CLASP-GRPO: Feasibility-Preserving Grouped Reinforcement Learning for LLM Combinatorial Optimization

Anonymous Author(s)
Affiliation
Address
email

Abstract

Large language models can generate solutions to combinatorial optimization problems from natural-language descriptions, but reinforcement learning for this setting remains limited by feasibility. We study *CLASP-GRPO*, a critic-free grouped method whose training credit is aligned with best-of-K evaluation: a group succeeds when at least one completion is feasible, and quality is judged among the feasible completions. The current implementation uses dense, tie-aware rank credit for successful groups and a lower-violation recovery surrogate for all-infeasible groups, together with taskwise projected controller updates. We evaluate a frozen current implementation on five combinatorial optimization tasks and keep canonical matched results separate from decoder-tuned development validation. The resulting evidence supports a reproducible comparison protocol, not a claim of exact historical-source reproduction, official FOARL reproduction, or statistical non-inferiority.

1 Introduction

Combinatorial optimization (CO) is a demanding test for large language models (LLMs). Solving a CO instance requires generating a discrete object that is both valid and high quality: it must satisfy task constraints and achieve a strong objective value. Recent work shows that decoder-only LLMs can map natural-language descriptions of CO instances directly to solutions through supervised fine-tuning (SFT) followed by reinforcement learning (RL) [Jiang et al., 2025]. ACCORD and StarJob illustrate related task-specific directions in feasibility-aware routing generation and LLM-based job shop scheduling [Abgaryan et al., 2025a,b]. The central challenge is therefore *feasibility-preserving optimization*.

The challenge is especially sharp under group-based policy optimization. Group Relative Policy Optimization (GRPO) compares multiple completions sampled from the same prompt and normalizes reward within the group rather than fitting a learned critic, so learning is driven by relative performance among candidates generated for the same instance [Shao et al., 2024, Mroueh, 2025]. In CO, hard prompts often produce groups in which every sampled completion is infeasible. On such groups, centered scalar reward can collapse to small penalty differences, even though the useful comparison is which infeasible sample is closer to feasibility without sacrificing too much solution quality.

We study *CLASP-GRPO* (Constrained Lagrangian Adaptive Structured Preference GRPO), a grouped RL method for this regime. CLASP keeps the grouped-sampling interface of GRPO, but aligns credit with the best-of-K deployment objective: successful groups compare feasible candidates by objective quality, while all-infeasible groups receive a lower-violation recovery signal.

Submitted to 40th Conference on Neural Information Processing Systems (NeurIPS 2026). Do not distribute.

Taskwise controller variables are updated from minibatch signed gaps. The method remains critic-free and prompt-grouped, but does not reduce every group to a single centered scalar reward.

Our claim is deliberately narrow: in LLM-based CO, grouped RL can work better when completions are compared through *prompt-local feasibility and quality trade-offs* rather than centered scalar reward alone. We test this claim on a unified five-task benchmark spanning TSP, OP, CVRP, MIS, and JSSP, using a shared decoder-only policy and a common natural-language interface. We also study a data-constrained supervised setting. Relative to Jiang et al., our SFT stage uses $10\times$ less supervised data, allowing us to test whether CLASP can recover competitive performance when fine-tuning data are limited. Unless otherwise noted, the training and evaluation datasets for these five tasks are taken from Jiang et al. [2025].

The paper makes four contributions. First, we document a grouped-RL implementation whose training credit is aligned with best-of-K feasible-set evaluation. Second, we present a unified empirical protocol that separates feasibility, best-feasible quality, and development decoder selection. Third, we connect the implementation to a constrained multi-task objective with dense task-specific costs while distinguishing controller statistics from the set utility used for credit. Fourth, we make the source, receipts, caveats, and regeneration status explicit so that the reported comparison can be audited rather than mistaken for an exact historical-source reproduction.

2 Related Work

We position CLASP at the intersection of learned combinatorial optimization, LLM-based direct solution generation, and grouped RL. In learned CO more broadly, prior neural approaches include autoregressive construction policies, attention-based solvers, multi-solution training, diffusion-style generators, and search-coupled systems such as NeuroLKH, DIMES, DIFUSCO, COMPASS, and DeepACO [Bengio et al., 2021, Vinyals et al., 2015, Kool et al., 2019, Kwon et al., 2020, Xin et al., 2021, Qiu et al., 2022, Sun and Yang, 2023, Chalumeau et al., 2023, Ye et al., 2023]. A related line learns components for exact or hybrid pipelines, including branching, neighborhood selection, and more unified learned solvers [Gasse et al., 2019, Wu et al., 2021, Wang et al., 2021, Liu et al., 2024, Drakulic et al., 2025, Pan et al., 2025]. Strong classical solvers also remain the main quality reference points for several tasks we study, including Concorde, LKH-style local search, hybrid genetic search, and standard OP/JSSP heuristic or constraint-based methods [Applegate et al., 2006, Helsgaun, 2017, Vidal, 2022, Vansteenwegen et al., 2011].

Among LLM-based CO papers, the closest baseline is Jiang et al. [2025]. Jiang et al. study end-to-end direct solution generation across multiple CO tasks using supervised fine-tuning and RL, and they also provide the five-task benchmark that we adopt here. Our empirical setting is intentionally aligned with theirs so that the comparison isolates a different question: not whether LLMs can be trained for CO at all, but how grouped RL should assign credit when prompt-level candidate sets mix feasible and infeasible solutions.

Two additional papers are especially relevant because they highlight task-specific feasibility handling. ACCORD [Abgaryan et al., 2025a] focuses on routing problems and uses a specialized autoregressive design for constraint-satisfying generation. StarJob [Abgaryan et al., 2025b] studies LLM-based job shop scheduling and emphasizes domain-specific scheduling structure. We discuss both because they reinforce the importance of feasibility-aware generation, but our goal is different: CLASP is a single grouped-RL training rule for a shared decoder-only policy across multiple CO tasks rather than a task-specific architecture or scheduler.

On the RL side, CLASP builds most directly on PPO and GRPO as grouped policy-optimization tools [Schulman et al., 2017, Shao et al., 2024]. We also draw conceptual motivation from constrained and primal–dual policy optimization, though we do not claim their strongest convergence guarantees [Achiam et al., 2017, Tessler et al., 2019, Stooke et al., 2020, Dai et al., 2024]. Relative to this literature, our contribution is narrower and more mechanistic: we study whether prompt-local comparisons between solution quality and feasibility yield a better grouped learning signal than centered scalar reward when feasibility is the main bottleneck.

3 Methodology

3.1 Problem Setting and Model Architecture

We consider a finite set of tasks

$$\mathcal{T} = \{\mathrm{TSP}, \mathrm{CVRP}, \mathrm{OP}, \mathrm{MIS}, \mathrm{JSSP}\}.$$

For each task $\tau \in \mathcal{T}$, a prompt x combines a task instruction with a serialized problem instance, and a decoder-only policy $\pi_\theta(\cdot \mid x)$ produces a structured textual completion such as a route, a set, or a schedule. We use a single shared language model across all tasks and introduce no task-specific neural heads; parameterization details are deferred to Appendix D.

A rollout group is formed by sampling K completions $y_1, \ldots, y_K$ from the same prompt. This grouped-sampling structure is inherited from GRPO and is fundamental to the method: CLASP does not add a learned critic, but changes how the group is compared internally.

All methods share the same prompt format, task-specific output schema, and SFT initialization. We first perform mixed-task SFT and then compare RL variants from that common starting point, so differences can be attributed to the grouped optimization rule rather than to representation or formatting changes. Full data and training details are deferred to Appendix D. This common setup keeps the comparison focused on optimization behavior rather than on architecture or prompt-format differences.

3.2 Task Rewards

Each sampled completion is assigned a task-specific objective reward and a lightweight structure reward. The structure reward captures syntactic or formatting regularity that is useful for stable learning but is not itself a feasibility certificate. The objective reward measures solution quality after parsing and task-specific recomputation. We write the primal score as

$$r_i = w_o\, o_i + w_s\, s_i, \tag{1}$$

where o_i and s_i denote the objective and structure rewards, respectively. Both terms are constructed as nonnegative rewards to be maximized across all tasks, so a value of zero denotes the lowest reward level rather than a zero-cost solution.

For theoretical exposition, it is convenient to distinguish an idealized thresholded-feasibility gate from the empirical reward implementation. In the idealized constrained objective,

$$o_i = \begin{cases} \tilde{o}_i, & \text{if the parsed completion is feasible,} \\ 0, & \text{otherwise,} \end{cases} \tag{2}$$

where $\tilde{o}_i$ is the ungated objective reward. When infeasible partial credit is enabled in the empirical system, we instead use

$$o_i = \begin{cases} \tilde{o}_i, & \text{if the parsed completion is feasible,} \\ \gamma_i \tilde{o}_i, & \text{otherwise,} \end{cases} \tag{3}$$

where $\gamma_i \in [0, 1]$ is the fraction of task-specific constraint checks satisfied by the parsed completion, with $\gamma_i = 0$ on parse failure. Disabling partial credit sets $\gamma_i = 0$ for every infeasible completion and recovers Eq. (2). This split keeps the reward design interpretable by separating solution quality from lightweight format regularization.

3.3 Dense Constraint Representation

For each completion y_i from task τ, the parser returns a normalized dense cost vector and a binary observability mask,

$$\mathbf{c}_i^{(\tau)} \in [0, 1]^{m_\tau}, \qquad \mathbf{m}_i^{(\tau)} \in \{0, 1\}^{m_\tau}, \tag{4}$$

where $c_{ij}^{(\tau)}$ is the normalized magnitude of violation type j, and $m_{ij}^{(\tau)}$ indicates whether that component is defined for the current sample. A key assumption, used later in both the population-feasibility

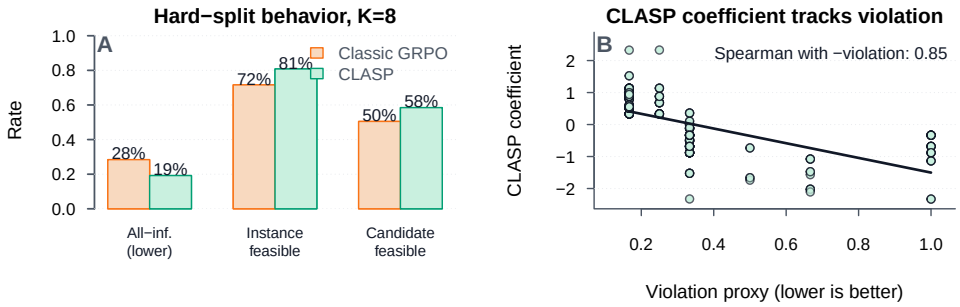


Figure 1: **Inference hard-group diagnostic.** Left: on the hard split with $K = 8$ candidates per prompt, CLASP reduces all-infeasible groups and improves instance- and candidate-level feasibility relative to classic GRPO. Right: on all-infeasible groups with observable violation variation, the CLASP grouped advantage is negatively associated with the violation proxy, indicating that lower-violation candidates receive larger grouped advantages. The violation proxy is $\max(0, 1-\gamma)$, where γ is the logged `constraint_fraction`.

discussion and the controller interpretation, is that the mask family is *complete* for the task semantics: whenever a violation type is semantically applicable to a sample, the corresponding mask entry must be active and the normalized cost must be computed. This assumption can fail in practice if the parser discards malformed fragments, if some violation categories are only partially instrumented, or if the serialization omits information needed to evaluate a constraint. The task-specific components are listed in Appendix F.

We use the exact positive violation score

$$v_i = \sum_{j=1}^{m_\tau} m_{ij}^{(\tau)} \max\left\{0, c_{ij}^{(\tau)} - \epsilon_j^{(\tau)}\right\},$$

only to define thresholded feasibility, so a completion is feasible iff $v_i = 0$ with respect to the task tolerances $(\epsilon_j^{(\tau)})_{j=1}^{m_\tau}$. The CLASP grouped advantage instead uses the controller-weighted positive gap

$$g_i(\boldsymbol{\lambda}^{(\tau)}) = \sum_{j=1}^{m_\tau} \lambda_j^{(\tau)} m_{ij}^{(\tau)} \max\left\{0, c_{ij}^{(\tau)} - \epsilon_j^{(\tau)}\right\},$$

while the controller is updated from the signed masked gap

$$h_{ij}^{(\tau)} = m_{ij}^{(\tau)}\left(c_{ij}^{(\tau)} - \epsilon_j^{(\tau)}\right),$$

whose minibatch mean provides the update direction for the projected controller. We assume throughout that the mask family is *complete* for the task semantics. Concretely, for every task τ, sample (x, y), and violation type j, if the semantics of (x, y) make violation type j applicable, then the corresponding mask entry satisfies $m_j^{(\tau)}(x, y) = 1$ and the normalized cost $c_j^{(\tau)}(x, y)$ is computed. Equivalently, no violation type that is semantically applicable to a sample is silently dropped by the mask. This separation matters because thresholded feasibility, grouped comparison, and dual adaptation use different signals. The dense-cost representation is therefore the mechanism that lets one shared policy express task-specific feasibility pressure in a uniform way. It is also a limitation: extending CLASP to a new CO family requires a parser that can expose semantically meaningful, normalized violation components at training scale. When those components are ambiguous, expensive to compute, or incomplete, the dense-cost controller becomes a weaker approximation to task feasibility. Figure 1 gives a compact inference-only diagnostic of the all-infeasible-group failure mode.

3.4 CLASP-GRPO

The current CLASP implementation is aligned with the deployment metric used in evaluation: best feasible solution in a group of K samples. Let q_i be a high-is-better objective score, $v_i \geq 0$ the total

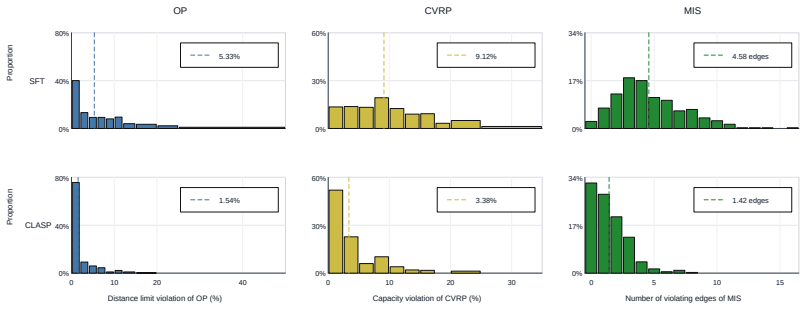


Figure 2: Constraint-violation distributions for SFT and CLASP on high-violation prompts across tasks. The dashed line in each panel marks the mean policy statistic for that problem. Lower violation mass indicates better feasibility behavior.

violation, and $z_i = \mathbf{1}[v_i = 0]$ the feasibility flag. For a group G, define the set utility

$$U(G) = \begin{cases} \text{smoothmax}_{i:z_i=1}(q_i), & \sum_i z_i > 0, \\ -\lambda_G, & \sum_i z_i = 0. \end{cases} \tag{5}$$

The exact leave-one-out credit is the marginal

$$\Delta_i^{\text{LOO}} = U(G) - U(G \setminus \{i\}). \tag{6}$$

The canonical training path uses the implementation's dense-rank estimator rather than this expensive exact marginal for every update. In successful mixed groups, every feasible completion is ranked above every infeasible completion, while feasible completions receive tie-aware objective rank credit and infeasible completions receive lower-violation rank credit. The resulting credit is centered within the group. In an all-infeasible group, the estimator uses a robust, centered lower-violation rank surrogate, with recovery pressure scaled by $1+\lambda_G$. Thus the implementation provides a signal for recovery even when the best-of-K set utility is flat before a feasible completion is observed.

The group multiplier λ_G is expanded from taskwise controller state and updated by projected signed-gap statistics at the configured update interval. These signed gaps are a controller signal, not the definition of the set utility. The canonical source configuration uses $K = 8$, dense-rank success credit, token-level importance sampling, no length penalty ($\alpha = 0$), no inference repair, and the recorded canonical scale parameters. The exact behavior is implemented in `compute_bon_aligned_advantages` and must be read together with the frozen source snapshot recorded in the paper manifest.

This definition is intentionally narrower than the historical manuscript's centered (r, g) mixture. The current results must not be described using the old logistic mixing coefficient, centered primal/gap equation, adaptive controller claim, or historical all-infeasible recovery percentages unless their source receipts are independently recovered. The retained diagnostic figures are legacy artifacts until regenerated from current receipts.

Our formal analysis focuses on a limited set of properties and is deferred to Appendix G. There, we show that population-level feasibility aligns with the dense-cost constraints under mask completeness, that reference-sample terms cancel under group centering, that the grouped advantage remains non-degenerate in a specific all-infeasible regime, and that the projected signed-gap controller satisfies fixed-point sign conditions. These results provide theoretical support for the mechanism underlying CLASP, but they do not imply unbiasedness, convergence, or exact optimization guarantees for the practical training algorithm.

3.5 Task-Adaptive Completion Length

The third component of CLASP is computational. Feasible outputs can differ dramatically in length across tasks: a TSP route is short, whereas a JSSP schedule can be much longer. A single global

Algorithm 1: One CLASP-GRPO training step.

Input: Current policy π_θ, old policy π_{old}, taskwise controller weights Λ, mixed-task prompt batch $\{x_b, \tau_b\}_{b=1}^{B}$

Output: Updated policy parameters and taskwise controller weights

Sample a mixed-task batch and generate K completions per prompt from π_{old};

Parse each completion and compute objective quality q_i, dense costs $\mathbf{c}_i^{(\tau)}$, masks $\mathbf{m}_i^{(\tau)}$, exact violation v_i, feasibility z_i, and componentwise signed gaps $h_{ij}^{(\tau)}$;

For each prompt group, form the best-of-K set utility and the dense-rank success or all-infeasible recovery credit described above;

Center the selected credit within the group to obtain A_i^{CLASP};

Apply the clipped sequence-level policy update using A_i^{CLASP};

Every n_λ primal steps, update the taskwise controller weights using Eq. (27);

Refresh each task's decoding budget from a rolling quantile of observed completion lengths;

completion cap therefore mismatches compute allocation to task structure. For each task τ, CLASP maintains a rolling buffer of decoded lengths and periodically sets

$$L_\tau \leftarrow \min\{L_{\max}, \max\{L_{\min}, Q_q(\text{buf}_\tau) + 32\}\}, \tag{7}$$

where Q_q is the empirical q-quantile. Before the buffer is sufficiently populated, the implementation leaves the task budget at the global cap $L_{\max}$. This mechanism does not change the optimization objective, but it changes how rollout budget is distributed across tasks and reduces unnecessary truncation or over-allocation. Its role is mainly practical: it helps a single training loop allocate generation budget more sensibly across tasks with very different output lengths.

3.6 Population Objective and Estimator Analysis

The dense-cost construction induces a constrained multi-task objective that motivates, but does not exactly characterize, the practical algorithm. For each task τ, let $x \sim \mathcal{D}_\tau$ denote a prompt and $y \sim \pi_\theta(\cdot \mid x)$ a sampled completion. Define

$$J_{\tau,x}(\theta) = \mathbb{E}_{y\sim\pi_\theta(\cdot|x)}\big[r^{(\tau)}(x,y)\big], \tag{8}$$

$$J_\tau(\theta) = \mathbb{E}_{x\sim\mathcal{D}_\tau}\big[J_{\tau,x}(\theta)\big], \tag{9}$$

and the nonnegative thresholded-feasibility moments

$$V_{\tau,j}(\theta) = \mathbb{E}_{x\sim\mathcal{D}_\tau,\, y\sim\pi_\theta(\cdot|x)}\Big[m_j^{(\tau)}(x,y)\max\Big\{0, c_j^{(\tau)}(x,y) - \epsilon_j^{(\tau)}\Big\}\Big]. \tag{10}$$

Because the integrand is nonnegative, the feasible set is the equality set $V_{\tau,j}(\theta) = 0$ for all active task components, equivalently written as $V_{\tau,j}(\theta) \le 0$. Under the mask-completeness assumption, this aligns the population constraints with thresholded feasibility rather than with cancellation in expectation. If $p_{\text{task}}(\tau)$ denotes the effective task-sampling distribution of the mixed-task loader, the corresponding multi-task Lagrangian is

$$\mathcal{L}(\theta,\Lambda) = \sum_{\tau\in\mathcal{T}} p_{\text{task}}(\tau)\left[J_\tau(\theta) - \sum_{j=1}^{m_\tau}\lambda_j^{(\tau)}V_{\tau,j}(\theta)\right]. \tag{11}$$

This sign convention corresponds to a primal maximization problem with equivalent inequalities $V_{\tau,j}(\theta) \le 0$: larger $\lambda_j^{(\tau)}$ penalizes positive violation in the inner maximization. CLASP should therefore be read as a grouped stochastic method motivated by this reference objective, not as an algorithm proved to optimize Eq. (11) exactly. Because the grouped advantage depends on prompt-local feasibility through α_G and on within-group normalization through $(\bar{r}_G, \bar{g}_G, \sigma_r^G, \sigma_g^G)$, the finite-$K$ estimator does not collapse to a single centered scalarization with fixed global weights. The controller also updates from signed masked gaps because they provide a stable directional signal even when many samples lie strictly inside the feasible region.

Define the signed surrogate constraint moments

$$G_{\tau,j}(\theta) = \mathbb{E}_{x\sim\mathcal{D}_\tau,\, y\sim\pi_\theta(\cdot|x)}\Big[m_j^{(\tau)}(x,y)\big(c_j^{(\tau)}(x,y) - \epsilon_j^{(\tau)}\big)\Big]. \tag{12}$$

The projected update in Eq. (27) is a controller defined on minibatch estimates of $G_{\tau,j}$. Proposition 3 characterizes the fixed points of that controller rather than exact optimization of Eq. (11). In particular, zeros of $G_{\tau,j}$ do not exclude cancellation between positive and negative deviations around $\epsilon_j^{(\tau)}$.

The exact positive violation score can be written as

$$v^{(\tau)}(x,y) = \sum_{j=1}^{m_\tau} m_j^{(\tau)}(x,y) \max\left\{0, c_j^{(\tau)}(x,y) - \epsilon_j^{(\tau)}\right\}. \tag{13}$$

Then thresholded feasibility is equivalent to $v^{(\tau)}(x,y) = 0$. Appendix G collects the formal statements: the population constraints align with thresholded feasibility under mask completeness, the grouped advantage is well-defined, reference-sample constants cancel under group centering, the grouped advantage remains non-trivial in a specific all-infeasible constant-primal regime, and the projected controller satisfies complementary-sign fixed-point conditions. The regularity assumptions needed for these results are deferred to the appendix.

4 Experiments

We evaluate the current implementation under a frozen matched development protocol and keep decoder tuning on a disjoint development slice. The canonical reproduction uses four A100 GPUs for training and one A100 GPU per evaluation job; it is not an exact historical-source reproduction. The standalone vanilla GRPO row in the legacy manuscript is not promoted to the current comparison without a validated receipt. Unless stated otherwise, the five-task data originate from Jiang et al. [2025]; exact slices, checkpoint lineage, decoding parameters, and caveats are recorded in the paper manifest.

Our benchmark covers five tasks: TSP, OP, CVRP, MIS, and JSSP. Unless noted otherwise, learned methods share the same base model family, prompt template, tokenizer, task mixture, and two-stage pipeline of mixed-task SFT followed by RL. We defer full training and data details to Appendix D.

At evaluation time, we recompute every objective value from the parsed discrete solution rather than from the model's self-reported score. We report candidate feasibility rate, instance feasibility rate, and mean/median best-feasible reference gap. Mean and median gaps are conditional on instances for which the method finds at least one feasible candidate; infeasible instances are captured by the feasibility columns and by Success@k rates. Because conditional gaps can favor methods that solve only easier instances, we do not use the gap column alone to rank methods when feasibility differs substantially; we interpret each gap together with its corresponding feasibility rate and use Success@k as an all-instance threshold metric. These metrics separate the ability to find any feasible solution from the ability to find a good one; gap definitions, tie-breaking rules, candidate budgets, and dataset diagnostics are given in Appendix D.

Table 1: Results on the five tasks shared with Jiang et al. [2025]: TSP, OP, CVRP, MIS, and JSSP. All results are on our OOD held-out set. Higher feasibility (fea.) and lower reference-value gap (opt.) are better, where the reference value is solver generated near optimal solution. The opt. columns are conditional on instances with at least one feasible candidate and should be read together with feasibility; infeasible instances receive no artificial gap value and count as failures in Success@k metrics. Under **This work**, BoN means best-of-N decoding with $N = 8$, and 2500k/250k denote the base SFT dataset size before RL (500,000/50,000 examples per task).

Method	TSP		OP		CVRP		MIS		JSSP	
	fea.	opt.	fea.	opt.	fea.	opt.	fea.	opt.	fea.	opt.
General-purpose Language Models										
GPT-4o	39%	33.79% ± 16.6	59%	55.19% ± 15.7	15%	76.62% ± 7.9	8%	11.70% ± 11.8	7%	97.85% ± 23.7
GPT-4o-mini	28%	53.38% ± 21.5	80%	70.70% ± 10.3	3%	68.10% ± 10.5	1%	0.00% ± 0	8%	212% ± 121
Claude-Sonnet	66%	24.53% ± 10.7	49%	34.62% ± 14.1	30%	38.34% ± 15.9	13%	12.51% ± 12.5	10%	90.00% ± 21.6
Claude-Haiku	45%	38.60% ± 16.9	26%	51.26% ± 14.9	4%	61.48% ± 12.1	2%	23.33% ± 6.7	9%	95.51% ± 22.2
DeepSeek-V3	73%	35.75% ± 15.4	50%	46.10% ± 13.4	21%	58.22% ± 26.8	5%	12.05% ± 12.9	52%	103.19% ± 26.9
Llama3.3-70B	50%	69.08% ± 31.4	27%	48.98% ± 14.6	31%	97.31% ± 69.3	8%	37.12% ± 29.5	29%	105.01% ± 24.5
Qwen2.5-72B	20%	36.89% ± 34.6	32%	49.36% ± 16.6	61%	180.91% ± 105	14%	29.56% ± 16.2	53%	103.90% ± 61.7
Reasoning Models										
GPT-o3-mini	91%	306% ± 258	8%	43.93% ± 11.5	50%	139% ± 39.7	66%	9.23% ± 8.4	20%	77.86% ± 37.8
GPT-o1	54%	276% ± 242	31%	40.90% ± 16.3	24%	154% ± 117	82%	8.03% ± 12.9	29%	81.90% ± 29.6
DeepSeek-R1	48%	70.99% ± 23.1	60%	40.54% ± 13.7	26%	30.46% ± 18.1	41%	1.60% ± 3.6	5%	26.29% ± 8.5
Prompt Strategies										
OPRO	83%	35.98% ± 3.6	85%	53.96% ± 14.3	21%	37.03% ± 18.3	7%	5.95% ± 7.3	65%	83.35% ± 25.5
LMEA	77%	265% ± 131	48%	66.18% ± 10.5	24%	61.24% ± 19.0	5%	25.0% ± 14.2	44%	83.19% ± 23.9
PHP	84%	33.84% ± 14.6	43%	36.08% ± 15.1	33%	58.11% ± 26.4	5%	11.67% ± 9.1	56%	104.04% ± 29.2
SGE	98%	29.66% ± 43.3	93%	24.49% ± 38.4	84%	36.14% ± 59.2	92%	3.62% ± 7.6	87%	38.58% ± 49.2
Base paper (Jiang et al.)										
Jiang et al. SFT 2500k	89%	2.30% ± 1.9	54%	2.32% ± 2.6	59%	6.02% ± 3.9	80%	1.71% ± 3.9	100%	11.01% ± 7.9
Base SFT+classicGRPO 2500k	91%	2.32% ± 2.2	92%	4.25% ± 2.9	80%	8.27% ± 5.6	83%	1.34% ± 3.3	100%	10.94% ± 7.3
Jiang et al. SFT+RL+BoN 2500k	100%	1.07% ± 0.9	100%	1.85% ± 1.7	100%	4.53% ± 3.5	94%	1.04% ± 3.4	100%	8.20% ± 6.3
This work										
SFT+BoN 250k	99.3%	2.26% ± 1.86	95.7%	1.54% ± 2.59	61.3%	5.32% ± 4.19	74.0%	1.45% ± 3.71	90.3%	9.08% ± 7.13
Vanilla GRPO+BoN 250k	100.0%	2.18% ± 1.54	99.7%	1.42% ± 1.73	97.0%	8.04% ± 4.65	87.3%	4.19% ± 7.15	59.0%	6.19% ± 5.01
Dual-primal GRPO+BoN 250k	100.0%	2.10% ± 1.54	95.3%	1.22% ± 1.69	92.0%	6.01% ± 3.99	88.3%	4.37% ± 7.50	90.3%	8.49% ± 6.19
CLASP+BoN (ours) 250k	**100%**	**1.01% ± 0.91**	98%	**1.18% ± 1.38**	**100%**	**4.18% ± 1.97**	**96%**	**1.04% ± 3.71**	**100%**	**8.19% ± 6.13**

Table 2: Comparison with problem-specific solvers (LKH for TSP, EA4OP for OP, HGS for CVRP, and Gurobi for MIS and JSSP), Gurobi, and CLASP. Results are reported as optimality gap(time), and "-" denotes an infeasible solution. The CLASP row uses the same best-of-8 setting as Table 1; its time includes decoding, parsing, rescoring, and feasible-candidate selection.

Method	TSP	OP	CVRP	MIS	JSSP
SOTA method	0.00%(0.3)	1.60%(0.3)	-0.19%(1.2)	0.00%(1.0)	2.45%(10.0)
Gurobi (1s)	5.76%(1.0)	49.42%(1.0)	–	2.28%(1.0)	41.34%(1.0)
Gurobi (10s)	1.82%(10.0)	21.96%(10.0)	35.09%(10.0)	0.46%(10.0)	2.45%(10.0)
CLASP	1.01%(7.34)	1.18%(5.28)	4.18%(7.96)	1.04%(3.28)	8.19%(11.72)

The broader reference comparison helps separate aggregate gains from distributional ones. Table 3 provides a size-stratified diagnostic for the five-task subset. For the local learned-method rows, the scale bins are computed from the same full-300 logs as Table 1; because opt. gaps are conditional on feasibility, aggregate gaps are recovered by feasible-count weighting over bins rather than by an unweighted average. The small/medium/large grouping follows the benchmark scale convention. For scheduling problems with jobs and machines, such as PFSP and JSSP, scale is defined by the job–machine grid: small covers 5×5 to 10×10, medium covers 10×10 to 15×15, and large covers 15×15 to 20×20. For all other graph, routing, and subset-selection problems, such as TSP, OP, CVRP, MIS, and MVC, scale is defined by the number of nodes: small covers 10–30 nodes, medium covers 40–60 nodes, and large covers 70–100 nodes. We move the matched Taillard-format JSSP comparison to Appendix E.

Table 3: Results by problem scale on the five-task subset shared with Jiang et al. [2025], evaluated on our OOD held-out set. Scheduling scales are defined by job–machine grids: small 5×5–10×10, medium 10×10–15×15, and large 15×15–20×20; non-scheduling scales are defined by node counts: small 10–30, medium 40–60, and large 70–100. Gap is the mean best-feasible reference gap within each scale bin, while Success@k is the percentage of all instances in the bin with a feasible candidate within $k\%$ of the reference. Lower gap and higher Success@k are better; aggregate rows and scale-stratified rows should only be compared when they are generated from the same evaluation protocol and candidate set.

Method	Small instances				Medium instances				Large instances			
	Gap	Success@1	Success@5	Success@10	Gap	Success@1	Success@5	Success@10	Gap	Success@1	Success@5	Success@10
TSP												
OR-Tools	0.82%	76%	96%	99%	2.59%	28%	86%	99%	3.59%	12%	80%	99%
NN	19.36%	3%	5%	19%	24.13%	0%	0%	1%	26.19%	0%	0%	2%
FI	2.27%	51%	82%	96%	4.41%	8%	64%	95%	4.86%	2%	53%	99%
ACO	1.98%	48%	88%	100%	17.98%	0%	1%	6%	36.69%	0%	0%	0%
Jiang et al.	0.14%	96%	100%	100%	0.70%	74%	100%	100%	1.34%	44%	100%	100%
Dual-primal GRPO	1.32%	55%	98%	100%	2.23%	18%	93%	100%	2.75%	10%	95%	100%
CLASP	0.62%	52%	100%	100%	1.09%	25%	95%	100%	1.32%	6%	97%	100%
OP												
Greedy	16.06%	4%	8%	25%	18.91%	0%	0%	5%	20.30%	0%	0%	3%
GI	9.07%	20%	37%	63%	11.91%	0%	12%	51%	13.63%	0%	7%	32%
Tsili	3.85%	21%	68%	96%	9.54%	0%	2%	55%	13.80%	0%	0%	8%
ACO	3.49%	30%	76%	94%	6.24%	1%	35%	89%	7.95%	0%	15%	74%
Jiang et al.	1.47%	54%	95%	99%	2.04%	26%	96%	100%	2.10%	27%	96%	99%
Dual-primal GRPO	0.70%	79%	91%	94%	1.13%	57%	93%	96%	1.83%	32%	92%	95%
CLASP	0.55%	79%	94%	95%	1.23%	57%	95%	96%	1.75%	37%	90%	93%
CVRP												
OR-Tools	3.60%	45%	69%	93%	7.87%	3%	24%	72%	8.84%	0%	15%	71%
Sweep	18.36%	8%	18%	32%	20.59%	0%	1%	8%	22.07%	0%	0%	3%
PS	3.95%	24%	72%	93%	5.67%	2%	50%	89%	6.12%	0%	41%	89%
ACO	2.52%	44%	81%	96%	17.07%	0%	2%	14%	29.49%	0%	0%	0%
Jiang et al.	1.70%	52%	90%	97%	4.57%	8%	59%	98%	7.24%	1%	19%	84%
Dual-primal GRPO	3.40%	27%	70%	95%	6.19%	2%	37%	80%	9.03%	0%	6%	48%
CLASP	2.57%	29%	66%	92%	3.99%	2%	44%	85%	5.98%	0%	10%	59%
MIS												
Degree	5.89%	57%	57%	67%	7.52%	32%	42%	64%	9.61%	20%	33%	59%
Greedy	2.56%	84%	84%	86%	2.79%	67%	71%	90%	3.36%	53%	60%	83%
Jiang et al.	0.38%	97%	98%	98%	1.05%	86%	87%	94%	2.29%	47%	57%	65%
Dual-primal GRPO	1.91%	84%	84%	89%	5.18%	54%	62%	77%	6.87%	31%	42%	49%
CLASP	0.44%	83%	83%	89%	1.16%	53%	61%	78%	1.52%	31%	45%	50%
JSSP												
SPT	19.58%	2%	4%	17%	25.32%	0%	0%	1%	27.35%	0%	0%	0%
FIFO	24.38%	2%	4%	12%	32.97%	0%	0%	1%	39.00%	0%	0%	0%
ATC	20.71%	0%	12%	15%	24.30%	0%	0%	1%	27.99%	0%	0%	0%
Jiang et al.	2.86%	32%	79%	98%	9.56%	0%	8%	60%	16.25%	0%	0%	4%
Dual-primal GRPO	4.44%	29%	61%	92%	7.88%	11%	34%	65%	15.05%	0%	0%	9%
CLASP	4.09%	27%	58%	87%	6.93%	5%	34%	69%	13.55%	0%	0%	12%

5 Conclusion

For LLM-based combinatorial optimization, the central challenge is not only to improve objective quality, but to do so without losing feasibility. Our results show that this requires a stronger within-group learning signal than centered scalar reward alone. CLASP-GRPO addresses this issue with an adaptive centered grouped advantage, a feasibility-conditional signed-gap controller, and task-adaptive decoding budgets. Across five combinatorial optimization tasks, it consistently improves feasibility-preserving generation over centered GRPO baselines and remains competitive with the closest prior LLM-based CO system under matched compute and candidate budgets. This comparison is especially notable because our SFT stage uses 10 times less supervised data than the reproduced Jiang et al. setting, allowing us to test how well CLASP learns under substantially more data-constrained supervision. More broadly, the results indicate that grouped reinforcement learning for constrained generation becomes substantially more effective when prompt-local comparisons are structured around feasibility and quality jointly, rather than around a single centered reward. CLASP is therefore a practical and lightweight method for improving feasibility-aware LLM optimization, though it is not a replacement for specialized solvers.

References

Henrik Abgaryan, Tristan Cazenave, and Ararat Harutyunyan. Accord: Autoregressive constraint-satisfying generation for combinatorial optimization with routing and dynamic attention, 2025a. URL https://arxiv.org/abs/2506.11052.

Henrik Abgaryan, Tristan Cazenave, and Ararat Harutyunyan. Starjob: Dataset for llm-driven job shop scheduling, 2025b. URL https://arxiv.org/abs/2503.01877.

Joshua Achiam, David Held, Aviv Tamar, and Pieter Abbeel. Constrained policy optimization. In *Proceedings of the 34th International Conference on Machine Learning*, volume 70, pages 22–31, 2017. URL https://proceedings.mlr.press/v70/achiam17a.html.

David L. Applegate, Robert E. Bixby, Vašek Chvátal, and William J. Cook. *The Traveling Salesman Problem: A Computational Study*. Princeton University Press, 2006.

Yoshua Bengio, Andrea Lodi, and Antoine Prouvost. Machine learning for combinatorial optimization: A methodological tour dhorizon. *European Journal of Operational Research*, 290 (2):405–421, 2021. ISSN 0377-2217. doi: https://doi.org/10.1016/j.ejor.2020.07.063. URL https://www.sciencedirect.com/science/article/pii/S0377221720306895.

Felix Chalumeau, Shikha Surana, Clément Bonnet, Nathan Grinsztajn, Arnu Pretorius, Alexandre Laterre, and Tom Barrett. Combinatorial optimization with policy adaptation using latent space search. In A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, and S. Levine, editors, *Advances in Neural Information Processing Systems*, volume 36, pages 7947–7959. Curran Associates, Inc., 2023. URL https://proceedings.neurips.cc/paper_files/paper/2023/file/18d3a2f3068d6c669dcae19ceca1bc24-Paper-Conference.pdf.

Josef Dai, Xuehai Pan, Ruiyang Sun, Jiaming Ji, Xinbo Xu, Mickel Liu, Yizhou Wang, and Yaodong Yang. Safe RLHF: Safe reinforcement learning from human feedback. In *The Twelfth International Conference on Learning Representations*, 2024. URL https://openreview.net/forum?id=TyFrPOKYXw.

Darko Drakulic, Sofia Michel, and Jean-Marc Andreoli. GOAL: A generalist combinatorial optimization agent learner. In *nternational Conference on Learning Representations*, 2025. URL https://openreview.net/forum?id=z2z9suDRjw.

Maxime Gasse, Didier Chetelat, Nicola Ferroni, Laurent Charlin, and Andrea Lodi. Exact combinatorial optimization with graph convolutional neural networks. In H. Wallach, H. Larochelle, A. Beygelzimer, F. d'Alché-Buc, E. Fox, and R. Garnett, editors, *Advances in Neural Information Processing Systems*, volume 32. Curran Associates, Inc., 2019. URL https://proceedings.neurips.cc/paper_files/paper/2019/file/d14c2267d848abeb81fd590f371d39bd-Paper.pdf.

Keld Helsgaun. *An Extension of the Lin-Kernighan-Helsgaun TSP Solver for Constrained Traveling Salesman and Vehicle Routing Problems: Technical report*. Roskilde Universitet, December 2017.

Xia Jiang, Yaoxin Wu, Minshuo Li, Zhiguang Cao, and Yingqian Zhang. Large language models as end-to-end combinatorial optimization solvers. In D. Belgrave, C. Zhang, H. Lin, R. Pascanu, P. Koniusz, M. Ghassemi, and N. Chen, editors, *Advances in Neural Information Processing Systems*, volume 38, pages 164787–164826. Curran Associates, Inc., 2025. URL https://proceedings.neurips.cc/paper_files/paper/2025/file/f0c6c93e1dbff84daf6082fef7e1a094-Paper-Conference.pdf.

Wouter Kool, Herke van Hoof, and Max Welling. Attention, learn to solve routing problems! In *International Conference on Learning Representations*, 2019. URL https://openreview.net/forum?id=ByxBFsRqYm.

Yeong-Dae Kwon, Jinho Choo, Byoungjip Kim, Iljoo Yoon, Youngjune Gwon, and Seungjai Min. Pomo: Policy optimization with multiple optima for reinforcement learning. In H. Larochelle, M. Ranzato, R. Hadsell, M.F. Balcan, and H. Lin, editors, *Advances in Neural Information Processing Systems*, volume 33, pages 21188–21198. Curran Associates, Inc., 2020. URL https://proceedings.neurips.cc/paper_files/paper/2020/file/f231f2107df69eab0a3862d50018a9b2-Paper.pdf.

Chang Liu, Zhichen Dong, Haobo Ma, Weilin Luo, Xijun Li, Bowen Pang, Jia Zeng, and Junchi Yan. L2p-MIP: Learning to presolve for mixed integer programming. In *The Twelfth International Conference on Learning Representations*, 2024. URL https://openreview.net/forum?id=McfYbKnpT8.

Youssef Mroueh. Reinforcement learning with verifiable rewards: Grpo's effective loss, dynamics, and success amplification, 2025. URL https://arxiv.org/abs/2503.06639.

Wenzheng Pan, Hao Xiong, Jiale Ma, Wentao Zhao, Yang Li, and Junchi Yan. UniCO: On unified combinatorial optimization via problem reduction to matrix-encoded general TSP. In *International Conference on Learning Representations*, 2025. URL https://openreview.net/forum?id=yEwakMNIex.

Ruizhong Qiu, Zhiqing Sun, and Yiming Yang. Dimes: A differentiable meta solver for combinatorial optimization problems. In S. Koyejo, S. Mohamed, A. Agarwal, D. Belgrave, K. Cho, and A. Oh, editors, *Advances in Neural Information Processing Systems*, volume 35, pages 25531–25546. Curran Associates, Inc., 2022. URL https://proceedings.neurips.cc/paper_files/paper/2022/file/a3a7387e49f4de290c23beea2dfcdc75-Paper-Conference.pdf.

John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms, 2017. URL https://arxiv.org/abs/1707.06347.

Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo. Deepseekmath: Pushing the limits of mathematical reasoning in open language models, 2024. URL https://arxiv.org/abs/2402.03300.

Adam Stooke, Joshua Achiam, and P. Abbeel. Responsive safety in reinforcement learning by pid lagrangian methods. In *International Conference on Machine Learning*, 2020. URL https://api.semanticscholar.org/CorpusID:220404280.

Zhiqing Sun and Yiming Yang. DIFUSCO: Graph-based diffusion solvers for combinatorial optimization. In *Advances in Neural Information Processing Systems*, 2023. URL https://proceedings.neurips.cc/paper_files/paper/2023/file/0ba520d93c3df592c83a611961314c98-Paper-Conference.pdf.

Chen Tessler, Daniel J. Mankowitz, and Shie Mannor. Reward constrained policy optimization. In *International Conference on Learning Representations*, 2019. URL https://openreview.net/forum?id=SkfrvsA9FX.

Pieter Vansteenwegen, Wouter Souffriau, and Dirk Van Oudheusden. The orienteering problem: A survey. *European Journal of Operational Research*, 209(1):1–10, 2011. ISSN 0377-2217. doi: https://doi.org/10.1016/j.ejor.2010.03.045. URL https://www.sciencedirect.com/science/article/pii/S0377221710002973.

Thibaut Vidal. Hybrid genetic search for the cvrp: Open-source implementation and swap* neighborhood. *Computers & Operations Research*, 140:105643, April 2022. ISSN 0305-0548. doi: 10.1016/j.cor.2021.105643. URL http://dx.doi.org/10.1016/j.cor.2021.105643.

Oriol Vinyals, Meire Fortunato, and Navdeep Jaitly. Pointer networks. In C. Cortes, N. Lawrence, D. Lee, M. Sugiyama, and R. Garnett, editors, *Advances in Neural Information Processing Systems*, volume 28. Curran Associates, Inc., 2015. URL https://proceedings.neurips.cc/paper_files/paper/2015/file/29921001f2f04bd3baee84a12e98098f-Paper.pdf.

Runzhong Wang, Zhigang Hua, Gan Liu, Jiayi Zhang, Junchi Yan, Feng Qi, Shuang Yang, Jun Zhou, and Xiaokang Yang. A bi-level framework for learning to solve combinatorial optimization on graphs. In M. Ranzato, A. Beygelzimer, Y. Dauphin, P.S. Liang, and J. Wortman Vaughan, editors, *Advances in Neural Information Processing Systems*, volume 34, pages 21453–21466. Curran Associates, Inc., 2021. URL https://proceedings.neurips.cc/paper_files/paper/2021/file/b2f627fff19fda463cb386442eac2b3d-Paper.pdf.

Yaoxin Wu, Wen Song, Zhiguang Cao, and Jie Zhang. Learning large neighborhood search policy for integer programming. In M. Ranzato, A. Beygelzimer, Y. Dauphin, P.S. Liang, and J. Wortman Vaughan, editors, *Advances in Neural Information Processing Systems*, volume 34, pages 30075–30087. Curran Associates, Inc., 2021. URL https://proceedings.neurips.cc/paper_files/paper/2021/file/fc9e62695def29ccdb9eb3fed5b4c8c8-Paper.pdf.

Liang Xin, Wen Song, Zhiguang Cao, and Jie Zhang. Neurolkh: Combining deep learning model with lin-kernighan-helsgaun heuristic for solving the traveling salesman problem. In M. Ranzato, A. Beygelzimer, Y. Dauphin, P.S. Liang, and J. Wortman Vaughan, editors, *Advances in Neural Information Processing Systems*, volume 34, pages 7472–7483. Curran Associates, Inc., 2021. URL https://proceedings.neurips.cc/paper_files/paper/2021/file/3d863b367aa379f71c7afc0c9cdca41d-Paper.pdf.

Haoran Ye, Jiarui Wang, Zhiguang Cao, Helan Liang, and Yong Li. Deepaco: Neural-enhanced ant systems for combinatorial optimization. In A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, and S. Levine, editors, *Advances in Neural Information Processing Systems*, volume 36, pages 43706–43728. Curran Associates, Inc., 2023. URL https://proceedings.neurips.cc/paper_files/paper/2023/file/883105b282fe15275991b411e6b200c5-Paper-Conference.pdf.

A Data Pipeline and SFT Details

Both training stages operate on a unified instance representation so that cross-task differences arise from the tasks themselves rather than from incompatible preprocessing. Each raw example is mapped into the common fields `instruction`, `input`, `output`, and `instance_json`. Duplicates are removed within each split, and task sizes are capped when needed to control mixture balance. If a task provides separate evaluation files, we eliminate exact within-task train/eval overlap. Otherwise, evaluation uses a held-out partition carved from the task's available training data, while any reserved evaluation files remain untouched until post-training comparison.

For the shared five-task comparison with Jiang et al. [2025], we use the same train and evaluation datasets on the overlapping tasks, but reduce the supervised stage to 50,000 SFT examples per task instead of the 500,000-per-task SFT setting used in the reproduced Jiang et al. configuration. The reference solutions in these datasets are solver-generated rather than model-generated. Specifically, **TSP** reference solutions are generated with the Lin–Kernighan–Helsgaun (LKH-3) solver and serve as optimal or best-known references for the 10–100 node instances used here; **OP** reference solutions are generated using the EA4OP genetic-algorithm-based solver; **CVRP** reference solutions are near-optimal solutions produced by LKH-3; **MIS** reference solutions are optimal solutions obtained with the Gurobi exact solver; and **JSSP** reference solutions are near-optimal solutions computed with the CP-SAT solver from Google OR-Tools using a 300-second time limit.

For SFT, each example is mapped to

$$\texttt{prompt}_i = \mathrm{Prompt}(\texttt{instruction}_i, \texttt{input}_i), \tag{14}$$

$$\texttt{completion}_i = \texttt{output}_i \oplus \texttt{eos}, \tag{15}$$

$$\texttt{text}_i = \texttt{prompt}_i \oplus \texttt{completion}_i. \tag{16}$$

For supervision, we align loss with the generated answer whenever tokenization preserves the prompt as an exact prefix of the full sequence; in those cases, optimization is performed on the completion tokens alone. If that alignment is broken by tokenization, we instead optimize over the full prompt–completion sequence to avoid masking errors. In multi-task runs, batches are constructed to maintain task diversity whenever more than one task is active, and the same mixture policy is retained during GRPO so that differences between stages are attributable to the learning objective rather than to a shift in task composition.

B Controller, Claims, and Experimental Details

B.1 Reward and Controller Details

The primal score combines a small structure reward with an objective reward. For every task,

$$\begin{aligned} s_i = \frac{1}{3}\Big(&\mathbf{1}[\text{required block exists}] \\ &+ \mathbf{1}[\text{parser returns a non-empty structure}] \\ &+ \mathbf{1}[\text{declared objective/makespan exists}]\Big), \end{aligned} \tag{17}$$

so $s_i \in [0, 1]$. This term rewards parser-compatible formatting rather than feasibility itself.

Objective quality is measured relative to the reference output in the RL data. Let δ_i denote the signed relative gap, positive when the candidate is worse than the reference and negative when it is better:

$$\delta_i = \begin{cases} \dfrac{\mathrm{pred}_i - \mathrm{ref}_i}{\mathrm{ref}_i}, & \text{minimization}, \\ \dfrac{\mathrm{ref}_i - \mathrm{pred}_i}{\mathrm{ref}_i}, & \text{maximization}. \end{cases} \tag{18}$$

We then map this gap to

$$\psi(\delta_i) = \begin{cases} \exp(-3\delta_i), & \delta_i \geq 0, \\ 1 + 0.5\tanh(-2\delta_i), & \delta_i < 0, \end{cases} \tag{19}$$

and add a declaration-match bonus $a_i \in \{0, 1\}$ when the model's stated objective agrees with the value recomputed from the parsed structure. The ungated objective reward is

$$\tilde{o}_i = 2\,(0.9\,\psi(\delta_i) + 0.1\,a_i)\,. \tag{20}$$

In the empirical study, we set $w_o = 1.0$ and $w_s = 0.3$ in $r_i = w_o o_i + w_s s_i$.

For the clipped objective used in training, we use a *length-normalized sequence surrogate ratio*. If T_i denotes the generated length,

$$\rho_i(\theta) = \exp\left(\frac{1}{T_i}\sum_{t=1}^{T_i}\left[\log \pi_\theta(y_{i,t} \mid x_i, y_{i,<t}) - \log \pi_{\text{old}}(y_{i,t} \mid x_i, y_{i,<t})\right]\right). \tag{21}$$

This quantity is the exponential of the average token-level log-likelihood ratio, equivalently the geometric mean of tokenwise likelihood ratios over the sampled sequence. It is therefore *not* the exact sequence importance ratio $\pi_\theta(y_i \mid x_i)/\pi_{\text{old}}(y_i \mid x_i)$; rather, it is a bounded-scale surrogate used for numerical stability across outputs of different lengths. CLASP and Dual-primal GRPO use this surrogate and disable additional reward scaling. The vanilla GRPO row in Table 1 follows the standard trainer objective, so it should be interpreted as a conventional system-level baseline; isolating the surrogate alone would require an additional sequence-surrogate vanilla GRPO ablation. Accordingly, the clipped objective below should be read as the practical training surrogate used in our constrained variants, not as a quantity inheriting the standard importance-sampling interpretation or PPO monotonic-improvement guarantees. The clipped surrogate term is

$$\ell_i(\theta) = -\min\left(\rho_i(\theta)A_i^{\text{CLASP}},\ \text{clip}(\rho_i(\theta), 1-\epsilon_{\text{low}}, 1+\epsilon_{\text{high}})\,A_i^{\text{CLASP}}\right), \tag{22}$$

optionally with a KL regularizer of weight κ_{KL}.

We use *Dual-primal GRPO* as the canonical name for the constrained GRPO baseline. It shares the same SFT initialization, prompt groups, dense cost components, length-normalized sequence ratio, and clipped surrogate family as CLASP, but it does not use the adaptive centered primal-gap grouped advantage in Eq. (6). Its sample advantage is

$$A_i^{\text{DP}} = \big(r_i - \bar{r}_G\big) - \sum_{j=1}^{m_\tau} \lambda_j^{(\tau)} h_{ij}^{(\tau)}, \tag{23}$$

with $h_{ij}^{(\tau)}$ denoting the signed masked gap introduced above; the clipped surrogate above is then evaluated with A_i^{DP} in place of A_i^{CLASP}. Thus the baseline centers only the primal reward within the prompt group and adds an uncentered controller-weighted signed-gap term, whereas CLASP centers both primal and controller-weighted positive-gap comparisons inside the adaptive grouped advantage.

Each task maintains its own controller $\boldsymbol{\lambda}^{(\tau)}$. It acts as a practical weighting mechanism for feasibility pressure rather than exact dual ascent for the population Lagrangian. Under the sign convention of Eq. (11), the update is a projected signed-gap controller. Because minibatches are mixed-task but controllers are task-specific, the controller statistics are computed separately for each task. Let $\mathcal{B}_t^{(\tau)}$ be the completions in the current controller window whose task is τ, and let $N_t^{(\tau)} = |\mathcal{B}_t^{(\tau)}|$. Every n_λ primal steps, the task-local masked batch statistics are aggregated as

$$\hat{h}_{t,j}^{(\tau)} = \frac{1}{N_t^{(\tau)}} \sum_{i \in \mathcal{B}_t^{(\tau)}} m_{ij}^{(\tau)}\big(c_{ij}^{(\tau)} - \epsilon_j^{(\tau)}\big), \tag{24}$$

$$\bar{f}_t^{(\tau)} = \frac{1}{N_t^{(\tau)}} \sum_{i \in \mathcal{B}_t^{(\tau)}} \mathbf{1}[v_i = 0]. \tag{25}$$

The controller then uses the feasibility-conditional step size

$$\eta_\lambda^{(t,\tau)} = \eta_0 \sqrt{\frac{1 - \bar{f}_t^{(\tau)} + \varepsilon}{\bar{f}_t^{(\tau)} + \varepsilon}} \tag{26}$$

Table 4: Evaluation questions and corresponding measurements. The table maps each empirical claim to the evidence reported in this manuscript, rather than to a complete possible ablation suite.

Claim	Evidence reported in this paper
CLASP-GRPO improves feasibility	Table 1 reports feasibility and conditional reference gaps for SFT+BoN, vanilla GRPO+BoN, Dual-primal GRPO+BoN, CLASP+BoN, and the Jiang et al. reference rows; Tables 2, 3, 10, and 11 add solver, scale-stratified, aggregate, and runtime context.
CLASP-GRPO improves best feasible quality	Table 1 reports mean best-feasible reference gaps with dispersion for BoN=8, separately from feasibility; Table 3 reports conditional gap and Success@$1/5/10$ by size bin; Tables 8 and 9 provide targeted Taillard JSSP and TSPLIB comparisons, including the reported TSPLIB BoN=8/64 distinction.
The proposed mechanism explains the gains	Mechanism evidence consists of the main hard-group diagnostic in Figure 1, the training-variant comparison in Appendix Figure 3, the SFT-vs-CLASP violation distributions in Figure 2, the appendix all-infeasible-group diagnostics in Figures 7 and 5, the mechanism dashboard in Figure 8, and the training-dynamics curves in Figure 9. Tables 5, 6, and 7, together with Appendix F, define the data, sequence-pressure diagnostics, training settings, adaptive length mechanism, dense costs, infeasible-sample credit, feasibility rule, and controller parameters used by those diagnostics.

and updates

$$\lambda_{t+1,j}^{(\tau)} = \Pi_{[0,\lambda_{\max}]}\left(\lambda_{t,j}^{(\tau)} + \eta_{\lambda}^{(t,\tau)}\hat{h}_{t,j}^{(\tau)}\right). \tag{27}$$

This rule reacts to the *current* policy rather than to an exponential moving average of past violations. The projection keeps the controller in a compact domain and the shared step size couples controller aggressiveness to the observed feasibility regime. As in the main text, this appendix-level description is intentionally algorithmic: it specifies the mechanism used in practice, not a general convergence guarantee for the overall non-convex training dynamics. For zero-tolerance components, the clipped costs have no negative interior margin and the corresponding controller weight can only increase or remain fixed, whereas positive-tolerance components can yield negative signed gaps and therefore allow the projected controller weight to decay.

CLASP also maintains a task-adaptive completion budget. For each task τ, a rolling window of recent decoded lengths is updated online; every R optimizer steps, the generation cap is reset to the clipped empirical quantile in Eq. (7). This leaves the reward definition unchanged, but it changes the effective compute allocation across tasks during RL.

C Claim Map and Experimental Setup

D Benchmark and Configuration Details

The primary training configuration uses a Qwen2.5-7B-Instruct base model with multi-node H100 GPU training, bfloat16 arithmetic, and LoRA adaptation. We keep the SFT stage fixed across methods and vary only the RL stage. In the reported five-task setup, the SFT stage uses 50,000 supervised examples per task, compared with 500,000 in the reproduced Jiang et al. reference setting. Table 7 reports the SFT initialization and the RL-stage hyperparameters that control the CLASP grouped advantage, controller dynamics, and adaptive length mechanism.

Some implementation flags and historical ablation logs retain early names such as `clasp_pareto_tau` and `clasp_pareto_anchor` for checkpoint and script compatibility. In the notation of this paper, `clasp_pareto_tau` is the transition-sharpness parameter β, and disabling the historical `clasp_pareto_anchor` flag disables the adaptive grouped advantage in Eq. (6); no Pareto anchor enters the mathematical grouped advantage.

Table 5: Benchmark suite used in this paper. The table summarizes the tasks, data volumes, and output schemas used in the supported experimental setup. Reported SFT train-set sizes correspond to the subsets actually used in the experiments, before within-split deduplication and train/validation partitioning; each task uses 50,000 supervised examples. The five-task setting trains and evaluates on TSP, OP, CVRP, MIS, and JSSP; MVC and PFSP appear only in legacy statistics files and are excluded from the experiments.

Task	Objective direction	SFT train	RL train	Eval	Output schema
TSP	minimize	50,000	3,200	300	`Route`, `Objective`
OP	maximize	50,000	5,000	300	`Route`, `Objective`
CVRP	minimize	50,000	2,000	300	`Routes`, `Objective`
MIS	maximize	50,000	3,200	300	`Set`, `Objective`
JSSP	minimize	50,000	3,200	300	`Schedule`, `Makespan`

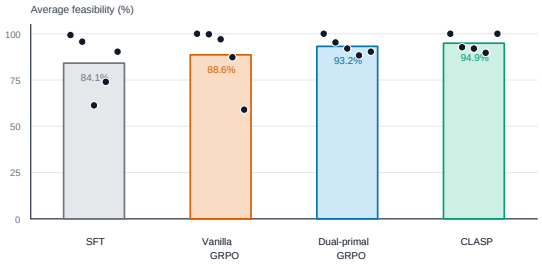


Figure 3: Average feasibility across the main training variants. Higher is better. The plotted comparison includes SFT, vanilla GRPO, Dual-primal GRPO, and CLASP; it should be read together with the main hard-group diagnostic in Figure 1, rather than as a complete factorial ablation suite.

Evaluation uses the same prompt template, parsers, and thresholded feasibility checks as training-time reward computation, but every candidate is rescored from its parsed discrete structure. For best-of-N decoding, we select the feasible candidate with the lowest recomputed reference gap for each instance; ties are broken by lower violation and then by the earliest sampled candidate. Instances with no feasible candidate contribute to feasibility and Success@k rates but are excluded from mean and median best-feasible gap summaries. We avoid assigning infeasible instances an arbitrary penalized optimality gap; when feasibility differs sharply, Success@k and feasibility are the fair all-instance comparisons, while best-feasible gap is a conditional quality diagnostic.

Table 6: Dataset and context diagnostics for the supported five-task setup. The table highlights sequence-length pressure and split overlap for the tasks used in the experiments. The token statistics aggregate only TSP, OP, CVRP, MIS, and JSSP, and the overlap count is based on a direct audit of the provided data splits.

Diagnostic	Value
Supported five-task combined sequences $>$ 4096 tokens	21.10%
Supported five-task combined sequences $>$ 8192 tokens	0.88%
Supported five-task combined sequences $>$ 12288 tokens	0.04%
TSP SFT-eval / GRPO-eval exact overlap	300
OP/CVRP/MIS/JSSP SFT-eval / GRPO-eval overlap	0

Table 7: Core training hyperparameters for the reported five-task CLASP configuration. The same rollout group size, batch settings, optimizer-facing overrides, and decoding cap are used for internal RL baselines; the length-normalized sequence surrogate is fixed across constrained variants unless a variant explicitly changes that component.

Category	Value
SFT initialization	Qwen2.5-7B-Instruct; bfloat16 arithmetic; no-reason Alpaca prompt style; seed 42; 50,000 supervised examples per task with evaluation ratio 0.05; LoRA rank 64, LoRA $\alpha = 64$, LoRA dropout 0; learning rate 2×10^{-4}; 5 epochs; context window / maximum sequence length 8192; per-device train/eval batch $2/1$, effective optimization batch 64 on 32 H100 GPUs.
RL model and adaptation	Initialized from the fixed SFT checkpoint; bfloat16 arithmetic; no-reason Alpaca prompt style; seed 42; full finetuning disabled; LoRA rank 64, LoRA $\alpha = 64$, LoRA dropout 0; learning rate 10^{-6}; 20 epochs.
Rollouts and surrogate	Rollout group size $K = 8$; per-device train/eval batch $4/4$; gradient accumulation 4, effective optimization batch 256 on 16 H100 GPUs; generation batch 512; maximum prompt length 7168; maximum completion length $L_{\max} = 1024$; context window 8192; constrained variants use the length-normalized sequence surrogate from Eq. (21) and PPO-style clipping as in Eq. (22), with no additional reward scaling.
Trainer defaults	Other trainer hyperparameters not overridden by the launchers, including optimizer type, scheduler, Adam coefficients, weight decay, gradient clipping, and PPO clipping constants, follow the TRL/Transformers defaults for the recorded TRL 0.24.0, Transformers 4.57.3, and Accelerate 1.12.0 environment.
CLASP grouped advantage and reward gate	Feasibility transition sharpness $\beta = 4.0$; within-group standard-deviation stabilizer $\varepsilon = 10^{-4}$; primal weight $w_o = 1.0$; structure weight $w_s = 0.3$; infeasible partial-credit gate from Eq. (3) enabled.
Controller	Base controller step size $\eta_0 = 0.0075$; controller refresh period $n_\lambda = 4$ primal optimizer steps; $\lambda_0 = 0$; $\lambda_{\max} = 1.0$; feasibility-conditioned step-size stabilizer $\varepsilon = 10^{-3}$.
Dense-cost thresholds	Global controller cost limit 0.03; violation tolerance 0.01; task-component tolerances are listed in Appendix F.
Adaptive length	Enabled; rolling-window size 1000; quantile $q = 0.99$; floor $L_{\min} = 128$; refresh period $R = 50$ optimizer steps.

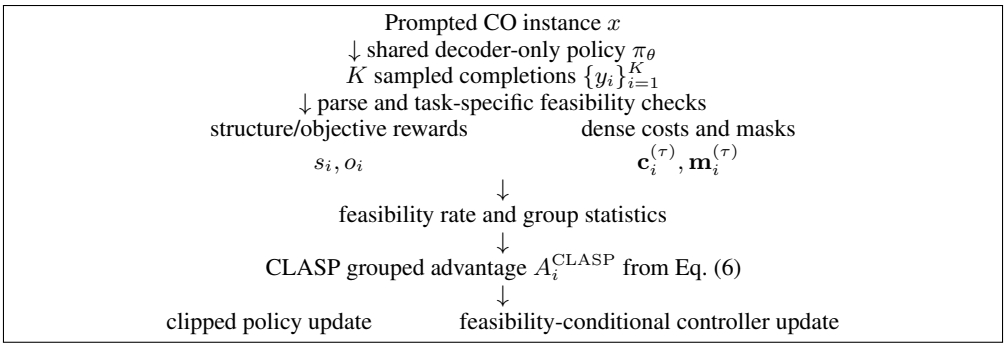


Figure 4: Overview of CLASP-GRPO, highlighting the adaptive centered within-group advantage, taskwise signed-gap controller, and adaptive decoding budget.

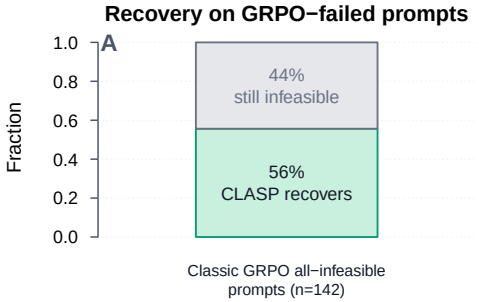


Figure 5: Recovery on prompts where classic GRPO produced no feasible candidate. The denominator is the 142 hard-split prompts for which all $K = 8$ classic GRPO candidates were infeasible; CLASP recovers at least one feasible candidate on 55.6% of them.

E Additional Reference Results and Diagnostics

Figure 4 provides a compact visual summary of the CLASP-GRPO training loop. Figures 3 and 5 move supporting mechanism summaries out of the main text: the former reports the training-variant feasibility comparison, and the latter shows recovery on prompts where classic GRPO produced no feasible candidate. Figure 6 adds a JSSP best-of-N ablation that varies the candidate budget to show how feasibility and reference-gap change with larger decoding pools. The additional mechanism diagnostics in Figures 7 and 8 show how often hard all-infeasible groups arise and how the grouped advantage and controller behave, while Figure 9 compares CLASP against Dual-primal GRPO over training.

F Task-Specific Dense Cost Components

The dense cost vectors used in this work are:

- **TSP**: `parse_failure`, `depot_boundary`, `invalid_node`, `duplicate_customer`, `missing_customer`, `interior_depot`.
- **CVRP**: `parse_failure`, `route_count_mismatch`, `depot_boundary`, `invalid_node`, `duplicate_customer`, `missing_customer`, `interior_depot`, `capacity_overflow`.
- **OP**: `parse_failure`, `start_at_depot`, `invalid_node`, `duplicate_visit`, `length_overflow`.
- **MIS**: `parse_failure`, `invalid_node`, `duplicate_node`, `edge_conflict`.

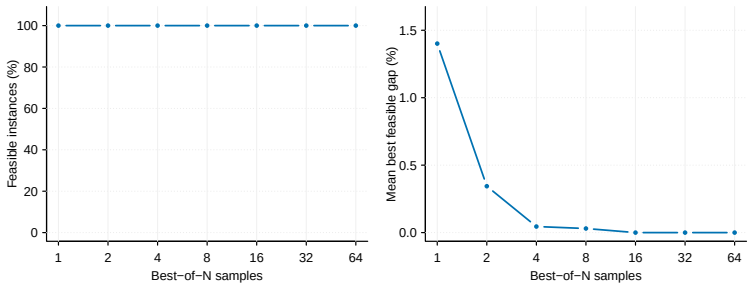


Figure 6: JSSP best-of-N ablation with varying candidate budget N. The plot reports how feasibility and conditional reference-value gap change as the best-of-N decoding budget increases from 1 to 64, providing appendix-level context for the BoN sensitivity of the JSSP results.

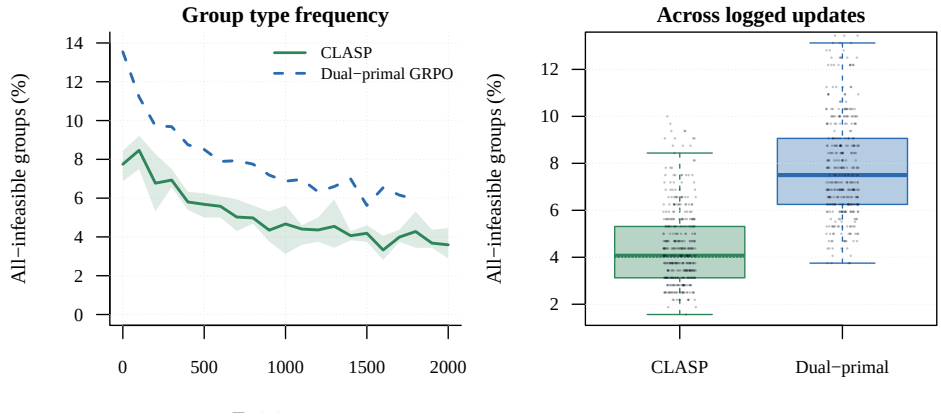


Figure 7: Frequency of hard prompt groups during training. Left: the fraction of all-infeasible groups over RL steps for CLASP and the Dual-primal GRPO baseline. Right: summary distribution of the all-infeasible-group rates across logged updates. The figure supports the motivating claim that hard prompts frequently induce all-infeasible groups and shows how the learned policies differ in that regime.

- **JSSP**: `parse_failure`, `machine_row_mismatch`, `row_length_mismatch`, `duplicate_job`, `missing_job`, `schedule_invalid`.

Every cost is clipped into $[0, 1]$, every mask is binary, and every sample must expose at least one active component.

- Tolerances use $\epsilon_j = 0$ for `parse_failure`, `depot_boundary`, `start_at_depot`, `invalid_node`, `duplicate_customer`, `missing_customer`, `duplicate_visit`, `duplicate_node`, `edge_conflict`, `machine_row_mismatch`, `row_length_mismatch`, `duplicate_job`, `missing_job`, `schedule_invalid`, and `interior_depot`; $\epsilon_j = 0.02$ for `route_count_mismatch` and `capacity_overflow`; and $\epsilon_j = 0.015$ for `length_overflow`.
- The binary feasibility flag is set to one only when the maximum positive component gap is at most 10^{-8}.
- The `co_default` weights are 5.0 for `parse_failure`, `missing_customer`, and `edge_conflict`; 4.0 for `invalid_node`; 3.5 for `depot_boundary`, `start_at_depot`, `machine_row_mismatch`, `row_length_mismatch`, and `duplicate_job`; 2.5 for `interior_depot`; 1.5 for `route_count_mismatch` and `duplicate_node`; 3.0 for `capacity_overflow` and `length_overflow`; 2.0 for `duplicate_visit`; and 4.5 for `missing_job`.

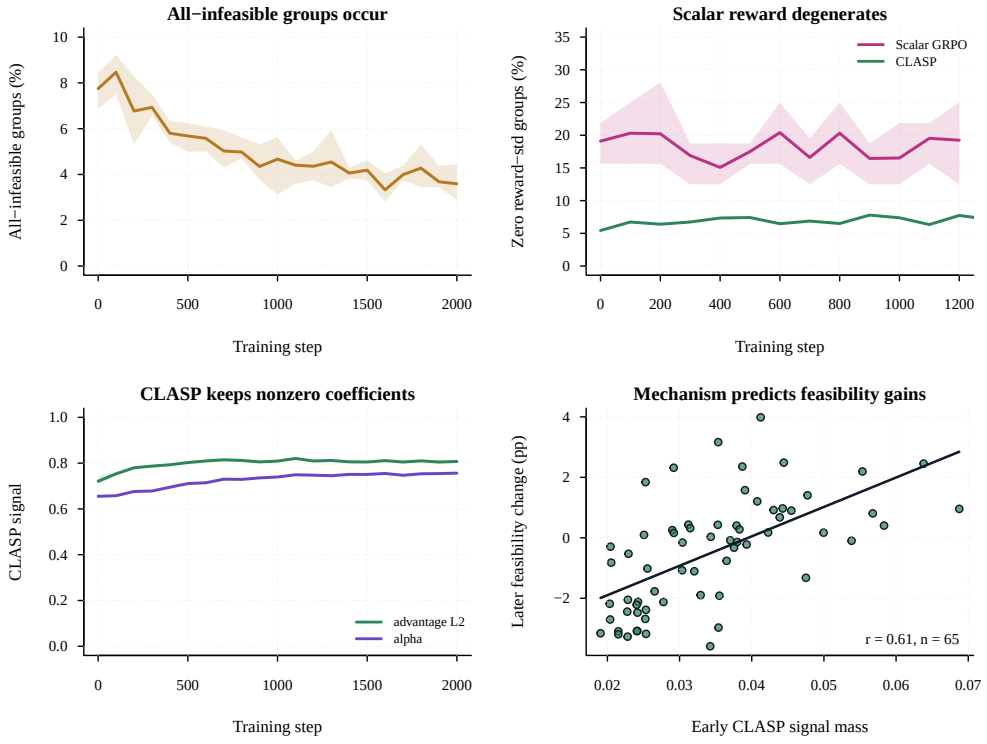


Figure 8: Mechanism diagnostics for CLASP. The panels summarize group-level failure modes and controller behavior, including all-infeasible-group prevalence, scalar-reward degeneracy, grouped-advantage evolution, and the relation between early CLASP gap reduction and later feasible-quality gains. Together they provide appendix-level evidence for the mechanism discussion in Section 3.

- Task-specific overrides use 6.0 for CVRP `missing_customer`, MIS `edge_conflict`, and JSSP `schedule_invalid`; 5.0 for CVRP `duplicate_customer` and JSSP `missing_job`; 4.0 for CVRP `capacity_overflow`, CVRP `depot_boundary`, OP `length_overflow`, and OP `start_at_depot`; and 1.0 for MIS `duplicate_node`.

Under the mask-completeness assumption, the controller therefore always observes some task-defined violation information, even for malformed outputs.

For monitoring only, we also log the scalar diagnostic summary

$$\text{constraint}_i = 1 - \frac{1}{|\{j : m_{ij}^{(\tau)} = 1\}|} \sum_{j : m_{ij}^{(\tau)} = 1} c_{ij}^{(\tau)}, \tag{28}$$

which is well-defined because every sample exposes at least one active component. This diagnostic is not used in either the CLASP grouped advantage or the controller update.

G Formal Arguments and Proofs

Proposition 1 (Population constraints align with thresholded feasibility)**.** *Fix a task τ and assume the mask family is complete for the task semantics. For each component j, define*

$$V_{\tau,j}(\theta) = \mathbb{E}_{x \sim \mathcal{D}_\tau,\, y \sim \pi_\theta(\cdot|x)} \left[m_j^{(\tau)}(x,y) \max\left\{0, c_j^{(\tau)}(x,y) - \epsilon_j^{(\tau)}\right\}\right]. \tag{29}$$

Then $V_{\tau,j}(\theta) \geq 0$ for every j, and the conjunction $V_{\tau,j}(\theta) = 0$ for all active task components is equivalent to almost-sure thresholded feasibility of the sampled completions with respect to the



Figure 9: Training-dynamics comparison between CLASP and Dual-primal GRPO. The panels track relative trends in feasibility, feasible quality, controller values, and related diagnostics over optimization. This figure is included in the appendix because it is useful for attribution and reviewer inspection, but too dense for the main paper.

dense costs and tolerances, i.e.,

$$\sum_{j=1}^{m_\tau} m_j^{(\tau)}(x,y)\max\Big\{0, c_j^{(\tau)}(x,y) - \epsilon_j^{(\tau)}\Big\} = 0 \quad \textit{for } \mathcal{D}_\tau \otimes \pi_\theta\textit{-almost every } (x,y). \tag{30}$$

Under mask completeness, this almost-sure condition coincides with thresholded feasibility for all semantically applicable violation types.

Proposition 2 (Centered CLASP grouped advantages are well-defined)**.** *For every finite prompt group $G = \{(r_i, g_i(\boldsymbol{\lambda}), v_i)\}_{i=1}^K$ with $K \geq 1$, finite r_i and $g_i(\boldsymbol{\lambda})$, well-defined feasibility indicators $\mathbf{1}[v_i = 0]$, $\beta \in (0,\infty)$, and $\varepsilon > 0$, the CLASP grouped advantages in Eq.* (6) *are finite and satisfy $\sum_{i=1}^K A_i^{\mathrm{CLASP}} = 0$.*

Remark 1 (Reference-sample cancellation). For any group-level reference index $a \in [K]$, including one selected from a Pareto frontier or any other deterministic rule, define the raw reference advantage

$$\widetilde{A}_i(a) = \alpha_G \frac{r_i - r_a}{\sigma_r^G + \varepsilon} + (1-\alpha_G)\frac{g_a(\boldsymbol{\lambda}) - g_i(\boldsymbol{\lambda})}{\sigma_g^G + \varepsilon}. \tag{31}$$

Its within-group centered form is independent of a:

$$\widetilde{A}_i(a) - \frac{1}{K}\sum_{k=1}^K \widetilde{A}_k(a) = \alpha_G \frac{r_i - \bar{r}_G}{\sigma_r^G + \varepsilon} + (1-\alpha_G)\frac{\bar{g}_G - g_i(\boldsymbol{\lambda})}{\sigma_g^G + \varepsilon} = A_i^{\mathrm{CLASP}}. \tag{32}$$

Thus no reference sample enters the mathematical grouped advantage in Eq. (6); historical implementation names involving "anchor" refer to enabling or disabling the whole centered grouped advantage.

Proposition 3 (Projected CLASP controller fixed-point conditions)**.** *Fix a task τ and freeze θ. Consider the deterministic controller obtained by replacing minibatch statistics with their expectations, and assume the signed moment $G_{\tau,j}(\theta)$ is finite:*

$$F_\tau(\theta) = \mathbb{E}_{x\sim\mathcal{D}_\tau,\, y\sim\pi_\theta(\cdot|x)}\big[\mathbf{1}[v(x,y)=0]\big], \tag{33}$$

$$\lambda_{t+1,j} = \Pi_{[0,\lambda_{\max}]}\left(\lambda_{t,j} + \eta_0\sqrt{\frac{1-F_\tau(\theta)+\varepsilon}{F_\tau(\theta)+\varepsilon}}\, G_{\tau,j}(\theta)\right), \tag{34}$$

where $\eta_0 > 0$, $\varepsilon > 0$, $\lambda_{\max} > 0$, and $\Pi_{[0,\lambda_{\max}]}$ denotes Euclidean projection onto $[0,\lambda_{\max}]$. Any fixed point $\lambda_j^\star$ satisfies the sign conditions

$$0 < \lambda_j^\star < \lambda_{\max} \implies G_{\tau,j}(\theta) = 0, \tag{35}$$

$$\lambda_j^\star = 0 \implies G_{\tau,j}(\theta) \leq 0, \tag{36}$$

$$\lambda_j^\star = \lambda_{\max} \implies G_{\tau,j}(\theta) \geq 0. \tag{37}$$

Theorem 1 (Non-degeneracy of the CLASP grouped advantage in the constant-primal all-infeasible regime)**.** *Let $G = \{(r_i, g_i(\boldsymbol{\lambda}), v_i)\}_{i=1}^K$ be a finite prompt group with $K \geq 2$ and finite real-valued primal scores and controller-weighted gaps. Assume that:*

1. *every completion in the group is infeasible, so $v_i > 0$ for all $i \in [K]$;*
2. *the primal scores are constant across the group, so $r_i = r_\circ$ for all $i \in [K]$;*
3. *the controller-weighted feasibility gaps are not constant across the group, so there exist $i, j \in [K]$ with $g_i(\boldsymbol{\lambda}) \neq g_j(\boldsymbol{\lambda})$;*
4. *The dense-rank and recovery estimators use finite positive stabilizers and bounded rank scores, so their centered group credits are finite.*

Then the centered grouped advantages $(A_i^{\mathrm{CLASP}})_{i=1}^K$ defined in Eq. (6) *are well-defined, sum to zero, and are not identically zero. By contrast, any mean-centered scalar-reward advantage based only on $(r_i)_{i=1}^K$ and proportional to $r_i - \bar{r}_G$, including the stabilized standardized quantity $(r_i - \bar{r}_G)/(\sigma_r^G + \varepsilon)$, is identically zero on the same group.*

Proof of Theorem 1. By Proposition 2, the CLASP grouped advantages are finite and sum to zero. Because every completion is infeasible, $f_G = 0$, so $\alpha_G = \sigma(-\beta/2) \in (0, 1)$ for every finite $\beta > 0$. Because the primal scores are constant across the group, $\bar{r}_G = r_\circ$, $\sigma_r^G = 0$, and the primal contribution to Eq. (6) is identically zero. Because the controller-weighted gaps are not constant, $\sigma_g^G > 0$.

Therefore the CLASP grouped advantages reduce to

$$A_i^{\mathrm{CLASP}} = (1 - \alpha_G)\frac{\bar{g}_G - g_i(\boldsymbol{\lambda})}{\sigma_g^G + \varepsilon}. \tag{38}$$

Since $(g_i(\boldsymbol{\lambda}))_{i=1}^K$ is not constant, at least one $g_i(\boldsymbol{\lambda})$ differs from $\bar{g}_G$. For that index, $A_i^{\mathrm{CLASP}} \neq 0$ because $1 - \alpha_G > 0$ and $\sigma_g^G + \varepsilon > 0$. By contrast, the primal scores are constant, so $r_i - \bar{r}_G = 0$ for every i. Hence every mean-centered scalar-reward advantage proportional to $r_i - \bar{r}_G$, including $(r_i - \bar{r}_G)/(\sigma_r^G + \varepsilon)$, is identically zero. Therefore A^{CLASP} is well-defined, zero-sum, and non-trivial, whereas the centered scalar baseline is degenerate on the same group. □

Proof of Proposition 1. Each integrand in $V_{\tau,j}(\theta)$ is nonnegative, so $V_{\tau,j}(\theta) \geq 0$. If $V_{\tau,j}(\theta) = 0$, then the nonnegative random variable

$$Z_{\tau,j}(x, y) = m_j^{(\tau)}(x, y) \max\left\{0, c_j^{(\tau)}(x, y) - \epsilon_j^{(\tau)}\right\} \tag{39}$$

has zero expectation and therefore vanishes for $\mathcal{D}_\tau \otimes \pi_\theta$-almost every (x, y). If this holds for every active component j, then the finite sum of these nonnegative terms also vanishes almost surely, which is exactly the displayed thresholded-feasibility condition. Conversely, if the summed thresholded-feasibility condition holds almost surely, then each nonnegative summand must vanish almost surely, implying $V_{\tau,j}(\theta) = 0$ for every j. Under mask completeness, every semantically applicable violation type is represented by an active mask entry and corresponding dense cost, so this almost-sure vanishing condition coincides with thresholded feasibility for all semantically applicable violation types. □

Proof of Proposition 2. The logistic sigmoid maps the finite argument $\beta(f_G - 1/2)$ to a finite $\alpha_G \in (0, 1)$. The denominators $\sigma_r^G + \varepsilon$ and $\sigma_g^G + \varepsilon$ are strictly positive because empirical standard deviations are nonnegative and $\varepsilon > 0$. Thus each grouped advantage in Eq. (6) is finite whenever the group scores are finite. Moreover,

$$\sum_{i=1}^K (r_i - \bar{r}_G) = 0, \qquad \sum_{i=1}^K (\bar{g}_G - g_i(\boldsymbol{\lambda})) = 0. \tag{40}$$

Multiplying these two zero sums by the group-constant factors in Eq. (6) gives $\sum_{i=1}^K A_i^{\mathrm{CLASP}} = 0$. □

Proof of Proposition 3. Fix a component j. The scalar factor

$$\kappa_\tau(\theta) = \eta_0 \sqrt{\frac{1 - F_\tau(\theta) + \varepsilon}{F_\tau(\theta) + \varepsilon}} \tag{41}$$

is finite and strictly positive: $F_\tau(\theta) \in [0, 1]$ because it is an expectation of an indicator, while $\eta_0 > 0$ and $\varepsilon > 0$. At a fixed point,

$$\lambda_j^\star = \Pi_{[0,\lambda_{\max}]}\left(\lambda_j^\star + \kappa_\tau(\theta) G_{\tau,j}(\theta)\right). \tag{42}$$

If $0 < \lambda_j^\star < \lambda_{\max}$, the projection is inactive at $\lambda_j^\star$, so Eq. (42) implies $\kappa_\tau(\theta) G_{\tau,j}(\theta) = 0$. Since $\kappa_\tau(\theta) > 0$, we obtain $G_{\tau,j}(\theta) = 0$.

If $\lambda_j^\star = 0$, Eq. (42) can hold only if the quantity inside the projection is non-positive. That is, $\kappa_\tau(\theta) G_{\tau,j}(\theta) \leq 0$. Positivity of $\kappa_\tau(\theta)$ then gives $G_{\tau,j}(\theta) \leq 0$.

If $\lambda_j^\star = \lambda_{\max}$, the quantity inside the projection must be at least $\lambda_{\max}$, which implies $\kappa_\tau(\theta) G_{\tau,j}(\theta) \geq 0$. Again $\kappa_\tau(\theta) > 0$, so $G_{\tau,j}(\theta) \geq 0$. This proves the stated complementary-sign conditions. □

H Appendix Discussion and Supplemental Tables

Tables 8, 9, 10, and 11 provide a compact appendix-level view of how the proposed method compares under four complementary lenses. Table 8 shows that the matched Taillard-format JSSP comparison remains competitive but still sensitive to evaluation protocol and candidate budget, so we interpret it as a targeted external reference rather than a standalone headline result. Figure 6 complements that table with a JSSP BoN ablation study over varying N, highlighting how larger decoding budgets affect feasibility and gap. Table 9 adds a TSPLIB TSP comparison against prior LLM-based methods and shows that increasing CLASP's best-of-N budget substantially narrows the gap.

Tables 10 and 11 summarize the broader positioning of CLASP across problems and runtime. Together they show that CLASP remains a fast learned policy and matches or improves the Jiang et al. best-of-N reference gaps across the five shared tasks under the current protocol. They also show that CLASP preserves the fast policy-decoding profile of the underlying SFT/RL model, even when it is modestly slower than the lightest open-weight general-purpose models on some tasks.

Table 8: Reference comparison on Taillard-format JSSP instances from Jiang et al. [2025]. Lower objective value and lower relative gap indicate better performance. Rows labeled "Jiang et al." correspond to entries labeled "Ours" in the source paper, and asterisks follow the source notation. Comparisons should be interpreted only under matched N and evaluation protocol."Adj.-swap polish" denotes a deterministic post-processing step that starts from the best feasible BoN schedule and greedily applies adjacent swaps in machine-order lists whenever the swap reduces the JSSP makespan.

Method	TA 15x15		TA 20x15		TA 20x20	
	obj.↓	opt.↓	obj.↓	opt.↓	obj.↓	opt.↓
Best-known solution	1228.9	–	1364.9	–	1617.3	–
SPT	1546.1	25.81%	1813.5	32.87%	2067.2	27.81%
FIFO	1657.4	34.87%	2008.4	47.15%	2297.1	42.03%
ATC	1586.8	29.12%	1794.3	31.46%	2114.1	30.72%
GA* 100x100	1583.8	28.88%	1847.1	35.23%	2304.2	42.47%
L2D greedy	1522.3	23.88%	1813.4	32.86%	2130.2	31.21%
L2D sample	1543.8	25.62%	1779.7	30.39%	2134.2	31.96%
StarJob*	–	19.68%	–	26.91%	–	33.12%
Jiang et al. (N = 8)	1403.6	14.22%	1653.7	21.16%	1994.2	23.30%
Jiang et al. (N = 64)	1368.2	11.34%	1600.5	17.26%	1940.9	20.01%
CLASP (N = 8)	1410.6	14.79%	1666.2	22.00%	2038.0	25.80%
CLASP (N = 64)	1402.9	14.16%	1643.4	20.32%	2009.8	24.06%
CLASP + adj.-swap polish ($N = 64$)	**1357.7**	**10.48%**	**1554.0**	**13.85%**	**1907.1**	**17.92%**

Table 9: Optimality gap comparison of LLM-based methods on TSPLIB instances. CLASP uses normalized-coordinate TSPLIB prompts; reported CLASP gaps are recomputed on the original TSPLIB coordinates with `EUC_2D` rounded distances.

Instance	AEL	ReEVO	MCTS-AHD	Jiang et al.	CLASP (BoN=8)	CLASP (BoN=64)
eil51.tsp	10.84%	6.50%	15.98%	2.02%	3.29%	1.64%
berlin52.tsp	12.91%	17.99%	7.08%	4.54%	4.07%	0.08%
st70.tsp	4.10%	6.42%	15.45%	0.00%	2.67%	0.15%
eil76.tsp	10.26%	7.34%	12.46%	2.91%	2.60%	1.12%
pr76.tsp	9.06%	12.17%	10.41%	3.87%	3.91%	2.48%
Average Gap	9.43%	10.08%	12.28%	2.69%	3.31%	1.09%

Table 10: Performance comparison across different optimization problems.

Method	TSP	OP	CVRP	MIS	JSSP
SOTA method	0.00%	1.60%	-0.19%	0.00%	2.45%
Gurobi (1s)	5.76%	49.42%	-	2.28%	41.34%
Gurobi (10s)	1.82%	21.96%	35.09%	0.46%	2.45%
Jiang et al.	1.07%	1.85%	4.53%	1.04%	8.20%
CLASP	1.01%	1.18%	4.18%	1.04%	8.19%

Table 11: Detailed average wall-clock solving time per problem instance for the evaluation setting used in the corresponding accuracy tables. Rows with best-of-N decoding, including CLASP, include decoding, parsing, rescoring, and feasible-candidate selection.

Method	TSP	OP	CVRP	MIS	JSSP
General-purpose Language Models					
GPT-4o	7.5s	4.8s	6.8s	3.7s	5.8s
GPT-4o-mini	5.1s	2.4s	5.5s	3.2s	9.0s
Claude-3.7-Sonnet	6.7s	4.4s	6.7s	3.6s	9.2s
Claude-3.5-Haiku	5.0s	4.3s	3.5s	4.9s	9.0s
DeepSeek-V3-671B	33.0s	22.8s	36.3s	20.3s	26.2s
Llama3.3-70B	2.6s	1.8s	2.2s	1.8s	3.1s
Qwen2.5-72B	13.5s	13.3s	13.6s	6.9s	22.1s
Reasoning Models					
GPT-o3-mini	57.2s	109.8s	60.5s	109.4s	45.9s
GPT-o1	118.7s	120.7s	72.1s	293.5s	98.2s
DeepSeek-R1	240.0s	521.3s	360.5s	402.9s	364.8s
Prompt Strategies					
OPRO	150.2s	123.7s	121.6s	103.1s	191.2s
LMEA	418.8s	276.7s	415.2s	241.9s	432.0s
PHP	96.1s	95.9s	104.2s	85.3s	125.2s
SGE	166.9s	234.3s	308.8s	163.7s	260.5s
Jiang et al.					
SFT	7.0s	4.6s	6.9s	3.2s	10.7s
SFT+RL	7.0s	4.6s	6.9s	3.2s	10.7s
SFT+RL+BoN	11.7s	10.8s	11.3s	4.0s	20.7s
CLASP	7.34s	5.28s	7.96s	3.28s	11.72s

I Additional Performance Comparison

Tables 10 and 11 collect the cross-problem summary and the runtime comparison in one place for quick reference.

NeurIPS Paper Checklist

1. **Claims**

   Question: Do the main claims made in the abstract and introduction accurately reflect the paper's contributions and scope?

   Answer: [Yes]

   Justification: Claims match the empirical and theoretical scope stated in the paper.

   Guidelines:

   - The answer [N/A] means that the abstract and introduction do not include the claims made in the paper.
   - The abstract and/or introduction should clearly state the claims made, including the contributions made in the paper and important assumptions and limitations. A [No] or [N/A] answer to this question will not be perceived well by the reviewers.
   - The claims made should match theoretical and experimental results, and reflect how much the results can be expected to generalize to other settings.
   - It is fine to include aspirational goals as motivation as long as it is clear that these goals are not attained by the paper.

2. **Limitations**

   Question: Does the paper discuss the limitations of the work performed by the authors?

   Answer: [Yes]

   Justification: Limitations are discussed in the conclusion and scope statements.

   Guidelines:

   - The answer [N/A] means that the paper has no limitation while the answer [No] means that the paper has limitations, but those are not discussed in the paper.
   - The authors are encouraged to create a separate "Limitations" section in their paper.
   - The paper should point out any strong assumptions and how robust the results are to violations of these assumptions (e.g., independence assumptions, noiseless settings, model well-specification, asymptotic approximations only holding locally). The authors should reflect on how these assumptions might be violated in practice and what the implications would be.
   - The authors should reflect on the scope of the claims made, e.g., if the approach was only tested on a few datasets or with a few runs. In general, empirical results often depend on implicit assumptions, which should be articulated.
   - The authors should reflect on the factors that influence the performance of the approach. For example, a facial recognition algorithm may perform poorly when image resolution is low or images are taken in low lighting. Or a speech-to-text system might not be used reliably to provide closed captions for online lectures because it fails to handle technical jargon.
   - The authors should discuss the computational efficiency of the proposed algorithms and how they scale with dataset size.
   - If applicable, the authors should discuss possible limitations of their approach to address problems of privacy and fairness.
   - While the authors might fear that complete honesty about limitations might be used by reviewers as grounds for rejection, a worse outcome might be that reviewers discover limitations that aren't acknowledged in the paper. The authors should use their best judgment and recognize that individual actions in favor of transparency play an important role in developing norms that preserve the integrity of the community. Reviewers will be specifically instructed to not penalize honesty concerning limitations.

3. **Theory assumptions and proofs**

   Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

   Answer: [Yes]

Justification: Assumptions and proofs are provided in Appendix G.

Guidelines:

- The answer [N/A] means that the paper does not include theoretical results.
- All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.
- All assumptions should be clearly stated or referenced in the statement of any theorems.
- The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.
- Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
- Theorems and Lemmas that the proof relies upon should be properly referenced.

4. **Experimental result reproducibility**

Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

Answer: [Yes]

Justification: The paper and appendix describe the setup, metrics, and comparisons needed to reproduce the main results.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- If the paper includes experiments, a [No] answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
- If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.
- Depending on the contribution, reproducibility can be accomplished in various ways. For example, if the contribution is a novel architecture, describing the architecture fully might suffice, or if the contribution is a specific model and empirical evaluation, it may be necessary to either make it possible for others to replicate the model with the same dataset, or provide access to the model. In general. releasing code and data is often one good way to accomplish this, but reproducibility can also be provided via detailed instructions for how to replicate the results, access to a hosted model (e.g., in the case of a large language model), releasing of a model checkpoint, or other means that are appropriate to the research performed.
- While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution. For example
  (a) If the contribution is primarily a new algorithm, the paper should make it clear how to reproduce that algorithm.
  (b) If the contribution is primarily a new model architecture, the paper should describe the architecture clearly and fully.
  (c) If the contribution is a new model (e.g., a large language model), then there should either be a way to access this model for reproducing the results or a way to reproduce the model (e.g., with an open-source dataset or instructions for how to construct the dataset).
  (d) We recognize that reproducibility may be tricky in some cases, in which case authors are welcome to describe the particular way they provide for reproducibility. In the case of closed-source models, it may be that access to the model is limited in some way (e.g., to registered users), but it should be possible for other researchers to have some path to reproducing or verifying the results.

5. **Open access to data and code**

Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

Answer: [Yes]

Justification: Code is provided with this submission as a zip and will be publically avaialbe after the reviews. The data used is from Jiang et al publically available source, discussed and cited the in Appendix and Main paper.

Guidelines:

- The answer [N/A] means that paper does not include experiments requiring code.
- Please see the NeurIPS code and data submission guidelines (`https://neurips.cc/public/guides/CodeSubmissionPolicy`) for more details.
- While we encourage the release of code and data, we understand that this might not be possible, so [No] is an acceptable answer. Papers cannot be rejected simply for not including code, unless this is central to the contribution (e.g., for a new open-source benchmark).
- The instructions should contain the exact command and environment needed to run to reproduce the results. See the NeurIPS code and data submission guidelines (`https://neurips.cc/public/guides/CodeSubmissionPolicy`) for more details.
- The authors should provide instructions on data access and preparation, including how to access the raw data, preprocessed data, intermediate data, and generated data, etc.
- The authors should provide scripts to reproduce all experimental results for the new proposed method and baselines. If only a subset of experiments are reproducible, they should state which ones are omitted from the script and why.
- At submission time, to preserve anonymity, the authors should release anonymized versions (if applicable).
- Providing as much information as possible in supplemental material (appended to the paper) is recommended, but including URLs to data and code is permitted.

6. **Experimental setting/details**

Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results?

Answer: [Yes]

Justification: Training setup, splits, metrics, and evaluation details are specified in the paper and appendix.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The experimental setting should be presented in the core of the paper to a level of detail that is necessary to appreciate the results and make sense of them.
- The full details can be provided either with the code, in appendix, or as supplemental material.

7. **Experiment statistical significance**

Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

Answer: [Yes]

Justification: Tables report values for key metrics such as feasibility and optimality gap and their corresponding standard deviations.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The authors should answer [Yes] if the results are accompanied by error bars, confidence intervals, or statistical significance tests, at least for the experiments that support the main claims of the paper.

- The factors of variability that the error bars are capturing should be clearly stated (for example, train/test split, initialization, random drawing of some parameter, or overall run with given experimental conditions).
- The method for calculating the error bars should be explained (closed form formula, call to a library function, bootstrap, etc.)
- The assumptions made should be given (e.g., Normally distributed errors).
- It should be clear whether the error bar is the standard deviation or the standard error of the mean.
- It is OK to report 1-sigma error bars, but one should state it. The authors should preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis of Normality of errors is not verified.
- For asymmetric distributions, the authors should be careful not to show in tables or figures symmetric error bars that would yield results that are out of range (e.g., negative error rates).
- If error bars are reported in tables or plots, the authors should explain in the text how they were calculated and reference the corresponding figures or tables in the text.

8. **Experiments compute resources**

   Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

   Answer: [Yes]

   Justification: Exact compute resources and runtime details are specified in the Main paper and in the Appendix.

   Guidelines:

   - The answer [N/A] means that the paper does not include experiments.
   - The paper should indicate the type of compute workers CPU or GPU, internal cluster, or cloud provider, including relevant memory and storage.
   - The paper should provide the amount of compute required for each of the individual experimental runs as well as estimate the total compute.
   - The paper should disclose whether the full research project required more compute than the experiments reported in the paper (e.g., preliminary or failed experiments that didn't make it into the paper).

9. **Code of ethics**

   Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics `https://neurips.cc/public/EthicsGuidelines`?

   Answer: [Yes]

   Justification: The research involves reinforcement learning algorithm designed for LLMs for combinatorial optimization using open-source data and benchmarks. It does not involve human subjects or obviously ethically sensitive applications, and we assume it conforms to the NeurIPS Code of Ethics.

   Guidelines:

   - The answer [N/A] means that the authors have not reviewed the NeurIPS Code of Ethics.
   - If the authors answer [No], they should explain the special circumstances that require a deviation from the Code of Ethics.
   - The authors should make sure to preserve anonymity (e.g., if there is a special consideration due to laws or regulations in their jurisdiction).

10. **Broader impacts**

    Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

    Answer: [No]

Justification: The paper focuses on the technical contributions on RL for LLMs and combinatorial optimization and does not include a specific discussion of broader positive or negative societal impacts.

Guidelines:

- The answer [N/A] means that there is no societal impact of the work performed.
- If the authors answer [N/A] or [No], they should explain why their work has no societal impact or why the paper does not address societal impact.
- Examples of negative societal impacts include potential malicious or unintended uses (e.g., disinformation, generating fake profiles, surveillance), fairness considerations (e.g., deployment of technologies that could make decisions that unfairly impact specific groups), privacy considerations, and security considerations.
- The conference expects that many papers will be foundational research and not tied to particular applications, let alone deployments. However, if there is a direct path to any negative applications, the authors should point it out. For example, it is legitimate to point out that an improvement in the quality of generative models could be used to generate Deepfakes for disinformation. On the other hand, it is not needed to point out that a generic algorithm for optimizing neural networks could enable people to train models that generate Deepfakes faster.
- The authors should consider possible harms that could arise when the technology is being used as intended and functioning correctly, harms that could arise when the technology is being used as intended but gives incorrect results, and harms following from (intentional or unintentional) misuse of the technology.
- If there are negative societal impacts, the authors could also discuss possible mitigation strategies (e.g., gated release of models, providing defenses in addition to attacks, mechanisms for monitoring misuse, mechanisms to monitor how a system learns from feedback over time, improving the efficiency and accessibility of ML).

11. **Safeguards**

Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pre-trained language models, image generators, or scraped datasets)?

Answer: [N/A]

Justification: The paper does not release high-risk models or datasets. The paper proposes a Reinforcement learning algorithm for LLMs for Combinatorial optimization. Neither the model nor the open source data used appear to create a high risk for misuse.

Guidelines:

- The answer [N/A] means that the paper poses no such risks.
- Released models that have a high risk for misuse or dual-use should be released with necessary safeguards to allow for controlled use of the model, for example by requiring that users adhere to usage guidelines or restrictions to access the model or implementing safety filters.
- Datasets that have been scraped from the Internet could pose safety risks. The authors should describe how they avoided releasing unsafe images.
- We recognize that providing effective safeguards is challenging, and many papers do not require this, but we encourage authors to take this into account and make a best faith effort.

12. **Licenses for existing assets**

Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

Answer: [Yes]

Justification: The paper properly cites the open source materials for existing assets like baseline methods and datasets (Jiang et al).

Guidelines:

- The answer [N/A] means that the paper does not use existing assets.
- The authors should cite the original paper that produced the code package or dataset.
- The authors should state which version of the asset is used and, if possible, include a URL.
- The name of the license (e.g., CC-BY 4.0) should be included for each asset.
- For scraped data from a particular source (e.g., website), the copyright and terms of service of that source should be provided.
- If assets are released, the license, copyright information, and terms of use in the package should be provided. For popular datasets, `paperswithcode.com/datasets` has curated licenses for some datasets. Their licensing guide can help determine the license of a dataset.
- For existing datasets that are re-packaged, both the original license and the license of the derived asset (if it has changed) should be provided.
- If this information is not available online, the authors are encouraged to reach out to the asset's creators.

13. **New assets**

    Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

    Answer: [Yes]

    Justification: We upload the code as a zip file, as a supplementary material which contains the readme.md file discussing the trainng details and useful commands for reproducibility. Aditionally the paper and the Appendix discuss the details for the training and inference phases of the method. We will release the codes publically after the review process.

    Guidelines:

    - The answer [N/A] means that the paper does not release new assets.
    - Researchers should communicate the details of the dataset/code/model as part of their submissions via structured templates. This includes details about training, license, limitations, etc.
    - The paper should discuss whether and how consent was obtained from people whose asset is used.
    - At submission time, remember to anonymize your assets (if applicable). You can either create an anonymized URL or include an anonymized zip file.

14. **Crowdsourcing and research with human subjects**

    Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

    Answer: [N/A]

    Justification: The paper does not involve crowdsourcing or human-subject studies.

    Guidelines:

    - The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
    - Including this information in the supplemental material is fine, but if the main contribution of the paper involves human subjects, then as much detail as possible should be included in the main paper.
    - According to the NeurIPS Code of Ethics, workers involved in data collection, curation, or other labor should be paid at least the minimum wage in the country of the data collector.

15. **Institutional review board (IRB) approvals or equivalent for research with human subjects**

    Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?

Answer: [N/A]

Justification: The paper does not involve crowdsourcing or human-subject studies, therefore IRB approval is not applicable.

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Depending on the country in which research is conducted, IRB approval (or equivalent) may be required for any human subjects research. If you obtained IRB approval, you should clearly state this in the paper.
- We recognize that the procedures for this may vary significantly between institutions and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the guidelines for their institution.
- For initial submissions, do not include any information that would break anonymity (if applicable), such as the institution conducting the review.

16. **Declaration of LLM usage**

Question: Does the paper describe the usage of LLMs if it is an important, original, or non-standard component of the core methods in this research? Note that if the LLM is used only for writing, editing, or formatting purposes and does *not* impact the core methodology, scientific rigor, or originality of the research, declaration is not required.

Answer: [Yes]

Justification: LLMs are a core component of the method and are described throughout the paper.

Guidelines:

- The answer [N/A] means that the core method development in this research does not involve LLMs as any important, original, or non-standard components.
- Please refer to our LLM policy in the NeurIPS handbook for what should or should not be described.